

College Timetable Viewer using React, HTML, and CSS

A Project Report

Submitted by

Shivek Gupta (24BCS10464)

Sharad Kumar Singh (24BCS11773)

Tanmay Bhardwaj (24BCS10727)

In partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING





BONAFIDE CERTIFICATE

Certified that this project report “**College Timetable Viewer using React, HTML, and CSS**” is the Bonafede work of “ Shivek Gupta (24BCS10464), Sharad Kumar Singh (24BCS11773) & Tanmay Bhardwaj (24BCS10727)” who carried out the project work under my/our supervision.

SIGNATURE

Dr. Sandeep Singh Kang

HEAD OF THE DEPARTMENT

BE-CSE

SIGNATURE

Er.Arshdeep Grover

SUPERVISOR

BE-CSE

ABSTRACT

The College Timetable Viewer is a modern web-based application developed to simplify the process of viewing, managing, and accessing academic schedules for students and faculty members. Traditional timetable systems often rely on static documents, notice boards, or manually updated spreadsheets, making it difficult for users to access accurate and organized schedule information. This project addresses these limitations by providing a responsive, interactive, and user-friendly timetable management platform built using React.js, HTML5, CSS3, and JavaScript.

The application offers a comprehensive set of features including weekly timetable view, daily timetable view, subject-wise filtering, faculty directory, classroom directory, attendance tracking, theme customization, advanced search functionality, responsive design, and print/export support. The system enables users to quickly locate academic information without manually browsing through lengthy schedules. The implementation of dynamic filtering and React state management ensures efficient performance and a seamless user experience.

One of the key strengths of the system is its modern user interface, which incorporates responsive layouts, glassmorphism design principles, dark and light themes, and mobile-friendly accessibility. The attendance tracker module allows students to monitor attendance percentages and estimate the number of classes required to achieve attendance goals. Additionally, the application supports Progressive Web App (PWA) concepts, enabling better performance and offline accessibility.

Despite the successful implementation, certain challenges were encountered during development, including timetable data organization, responsive layout optimization, state management, and maintaining consistency across different devices and browsers. These challenges were addressed through modular React components, centralized data structures, CSS media queries, and systematic testing procedures.

The future scope of the project includes the integration of AI-based timetable assistants, real-time synchronization, cloud database support, Google Calendar integration, push notifications, mobile application development, and machine learning-based timetable scheduling. These enhancements can further transform the application into a complete academic management ecosystem.

In conclusion, the College Timetable Viewer successfully demonstrates the practical application of modern web development technologies in solving real-world academic scheduling problems. The system provides an efficient, reliable, and visually appealing solution that improves timetable accessibility, enhances user productivity, and offers a strong foundation for future educational management systems.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who contributed to the successful completion of this project titled "**College Timetable Viewer Using React, HTML, CSS and JavaScript.**"

First and foremost, we would like to extend our heartfelt thanks to **Mr. Arshdeep Grover**, whose valuable guidance, continuous support, and expert suggestions played a vital role throughout the development of this project. His encouragement and technical insights helped us overcome various challenges and significantly improved the quality of our work.

We are also thankful to **Chandigarh University** for providing the necessary infrastructure, learning resources, laboratory facilities, and academic environment required for the successful execution of this project. The opportunities and support provided by the university enabled us to transform our ideas into a functional and practical web-based application.

We would also like to acknowledge our classmates and peers for their constructive feedback, helpful discussions, and encouragement during the project development process. Their suggestions helped us improve the functionality, design, and usability of the application.

Special thanks are extended to our family members and friends for their continuous motivation, patience, and emotional support throughout the project duration. Their confidence in our abilities inspired us to complete this work successfully.

Finally, we would like to thank all those who directly or indirectly contributed to the completion of this project. Without their support, guidance, and encouragement, this project would not have been possible.

Shivek Gupta (24BCS10464)

Sharad Kumar Singh (24BCS11773)

Tanmay Bhardwaj (24BCS10727)

(Student, B.E. Computer Science & Engineering)

Chandigarh University

Table of Contents

Chapter No.	Topic	Page No.
Certificate	Certificate	i
Abstract	Abstract	ii
List of Figures	List of Figures	iii
List of Tables	List of Tables	iv
Chapter 1	Introduction	1
1.1	Introduction to Timetable Management Systems	1
1.2	Background of the Study	2
1.3	Need for Digital Timetable Solutions	3
1.4	Problem Statement	4
1.5	Objectives of the Project	5
1.6	Scope of the Project	6
1.7	Benefits of the Proposed System	7
1.8	Organization of the Report	8
Chapter 2	Literature Review	9
Chapter 3	Requirement Analysis	14
Chapter 4	System Analysis and Design	19
Chapter 5	Technology Stack	25
Chapter 6	System Development and Implementation	30
Chapter 7	Algorithms and Logic	37
Chapter 8	User Interface Design	43
Chapter 9	Source Code Explanation	49
9.1	index.html	50
9.2	main.jsx	51

Chapter No.	Topic	Page No.
9.3	App.jsx	53
9.4	React Components	58
9.5	Search & Filter Module	61
9.6	Attendance Tracker	63
9.7	Theme Management	65
9.8	CSS Styling System	67
9.9	AI Assistant Module	70
9.10	Export & Print Module	72
Chapter 10	Testing and Validation	75
Chapter 11	Results and Discussion	80
Chapter 12	Future Enhancements	86
Chapter 13	Conclusion	91

CHAPTER 1: INTRODUCTION

1.1 Introduction to Timetable Management Systems

A timetable management system is an essential tool used in educational institutions to organize and display academic schedules in a clear, structured, and accessible manner. It helps students, faculty members, and administrators manage class timings, subject allocations, classroom assignments, and faculty responsibilities without confusion. In colleges and universities, where multiple classes, departments, laboratories, and faculty members operate simultaneously, the need for an efficient timetable system becomes highly important.

Traditionally, timetables were maintained manually on paper notice boards or circulated as printed sheets. Although this method was simple, it had several limitations such as difficulty in updating changes, poor accessibility, lack of search options, and inconvenience in handling large amounts of schedule data. With the advancement of web technologies, timetable management has evolved into digital platforms that provide instant access, dynamic filtering, responsive design, and improved usability.

The proposed project, **College Timetable Viewer using React, HTML, and CSS**, is a modern web-based solution designed to overcome the shortcomings of traditional timetable systems. It provides a user-friendly interface that allows users to view both weekly and daily schedules, search for subjects, filter by faculty or classroom, and navigate through timetable information with ease. The application focuses on clarity, responsiveness, and interactivity, making it suitable for everyday use in academic environments. The project also includes additional features such as dark/light theme support, attendance tracking, dashboard widgets, and print/export support, which improve the overall user experience.

.

1.2 Background of the Study

In educational institutions, time is one of the most valuable resources, and its proper management directly affects academic performance and institutional efficiency. Timetables act as the backbone of the academic calendar by ensuring that lectures, practicals, seminars, and other activities are arranged in a systematic order. A well-planned timetable reduces scheduling conflicts, prevents duplication of resources, and helps maintain discipline in teaching and learning activities.

Earlier, timetable preparation and distribution were mostly manual. Faculty coordinators often prepared timetables using spreadsheets or handwritten charts, and students relied on notice boards or printed copies to know their schedules. However, as institutions grew larger and more departments were introduced, the timetable structure became more complex. Manual systems could no longer handle frequent changes such as faculty substitutions, room changes, and special events.

The rise of web applications and front-end frameworks has made it possible to design efficient timetable systems that are interactive and easy to maintain. React.js, in particular, provides component-based architecture and state-driven updates, which are highly suitable for managing dynamic timetable data. HTML and CSS contribute to the structure and visual design of the interface, allowing developers to build clean, responsive, and accessible web pages. In this project, these technologies are combined to create a smart timetable viewer that supports

various user needs in a modern academic setting. The report shows that the application includes weekly and daily timetable views, filtering features, faculty and classroom directories, theme options, and printable output screens.

.

1.3 Need for Digital Timetable Solutions

The need for digital timetable solutions has increased significantly due to the growing complexity of academic operations. In a college environment, students and faculty members often require quick access to schedule information. A digital system provides several advantages over manual methods, such as instant updates, searchability, remote accessibility, and improved organization.

A traditional timetable is static in nature. Once it is printed or displayed, any change in lecture timing, classroom assignment, or faculty allocation requires a new version to be prepared and redistributed. This process consumes time and may create confusion if outdated copies continue to circulate. A digital timetable viewer solves this problem by allowing data to be updated centrally and displayed immediately to all users.

Another important reason for digital transformation is accessibility. Students today use mobile phones, tablets, and laptops for academic purposes. A timetable system that works across devices ensures that users can check schedules whenever needed, whether they are at home, in class, or on campus. The responsive design mentioned in the project report supports this requirement by making the interface mobile-friendly and adaptable to different screen sizes. The system also includes search and filter functionalities, enabling users to find relevant subjects, faculty members, or classrooms quickly. These features make the timetable more practical and efficient in daily academic life.

.

Digital timetable systems also improve communication. For example, if a classroom changes or a lecture is rescheduled, the update can be reflected in the system without delay. This reduces misunderstandings and ensures that students and faculty remain informed. In addition, advanced features like attendance tracking, dashboard summaries, and print/export support make the system more than just a timetable display tool; it becomes a complete academic management interface.

1.4 Problem Statement

Many colleges still face difficulties in managing timetables effectively due to outdated or poorly organized systems. Students often struggle to locate accurate class schedules, especially when there are multiple departments, elective subjects, practical sessions, and faculty changes. Faculty members may also find it difficult to communicate schedule updates clearly to all students in time.

The main problem addressed by this project is the lack of a centralized, user-friendly, and responsive timetable viewer that can display academic schedules in a clear format. Manual systems are often slow, less flexible, and prone to errors. They do not provide interactive features like filtering by subject, faculty, or classroom, and they usually fail to support modern devices and user expectations.

Another problem is that timetable information is frequently scattered across different sources such as notice boards, WhatsApp messages, printed charts, or departmental groups. This fragmentation creates confusion and increases the chance of students missing important classes. Furthermore, static timetable formats do not provide real-time navigation or personalized views, which are now considered essential in digital academic tools.

This project addresses these challenges by developing a single web-based platform where timetable information can be organized, displayed, and accessed efficiently. According to the project report, the system is designed to provide daily and weekly schedules, subject details, faculty information, classroom allocation, and filtering options through an intuitive interface. This directly solves the issues of accessibility, clarity, and flexibility in timetable management.

.

1.5 Objectives of the Project

The primary objective of this project is to develop a modern and efficient timetable viewer that helps students and faculty members access academic schedules quickly and accurately. The system should present timetable information in a clean and organized format while offering useful features that improve usability.

The project has several specific objectives. First, it aims to provide both weekly and daily timetable views so that users can switch between broad and detailed schedule perspectives. Second, it intends to implement search and filtering options so that users can locate specific subjects, faculty names, or rooms with ease. Third, it seeks to improve accessibility by creating a responsive interface that works smoothly on desktops, tablets, and mobile devices.

Another objective is to enhance user experience through a visually appealing design. The report shows that the system includes modern UI elements such as glassmorphism styling, theme support, and dashboard widgets. These elements make the timetable more engaging and easier to use. The project also aims to introduce advanced functionality such as attendance tracking and a goal calculator, which adds academic value beyond simple timetable display.

.

In addition, the project is designed to demonstrate the practical use of React, HTML, and CSS in a real-world educational application. It helps illustrate how front-end technologies can be used to build interactive systems that combine structure, functionality, and design. By completing this project, the developer gains experience in React component design, state management, dynamic rendering, responsive layout creation, and theme switching.

1.6 Scope of the Project

The scope of the project covers the design and development of a web-based college timetable viewer for academic use. The application is intended to display timetable data in an accessible and interactive way for students, faculty members, and administrators. It supports daily and weekly views, subject filtering, faculty and classroom details, navigation controls, search features, and responsive layouts.

The system is mainly focused on front-end development, where timetable data is stored in structured objects and rendered dynamically using React components. It does not require a heavy backend infrastructure for basic operation, which makes it suitable as a small to

medium-scale academic project. However, the architecture can be extended in the future to include backend integration, databases, authentication, and cloud synchronization.

The report also shows that the project includes features such as offline caching, PDF/print export, theme selection, and a dashboard overview. These features broaden the scope of the application by making it more practical and reusable in different academic settings. The timetable viewer can be used in departments, classrooms, computer labs, or even institution-wide portals. Because the design is modular, additional features like notifications, calendar sync, and collaborative updates can be added later without changing the core structure.

.

1.7 Benefits of the Proposed System

The proposed timetable viewer offers multiple benefits compared to traditional timetable management methods. The first major benefit is accessibility. Users can view their schedules anytime and from any device with a browser. This eliminates the dependence on notice boards or printed timetables.

The second benefit is clarity. Timetable information is presented in a structured and color-coded format, which makes it easier to understand. Weekly and daily views help users visualize their schedules in different ways, depending on their needs. The inclusion of subject-wise filters, faculty directories, and classroom lists further improves organization.

The third benefit is efficiency. Since users can search and filter timetable data instantly, they spend less time looking for information. This is especially useful in large colleges where schedules change often and users need immediate updates. The attendance tracker and goal calculator described in the report also provide additional academic value by helping students monitor attendance targets.

.

Another benefit is responsiveness. The system is designed using modern CSS techniques, making it compatible with different screen sizes. This ensures that the timetable remains usable on smartphones, tablets, laptops, and desktop screens. The dark/light theme support improves comfort and personalization, while the print/export functionality allows users to generate hard copies when necessary.

From a development perspective, the project is also beneficial because it demonstrates practical implementation of React-based UI design, state handling, and dynamic rendering. It serves as a strong academic example of how modern web technologies can be used to solve real-world scheduling problems.

1.8 Organization of the Report

This project report is organized in a systematic manner so that each part of the work can be easily understood. Chapter 1 introduces the project, its background, need, problem statement, objectives, scope, and benefits. It provides the foundation for the rest of the report.

Chapter 2 presents the literature review and discusses earlier systems and related approaches to timetable management. Chapter 3 explains the requirement analysis, including software requirements, hardware requirements, and feasibility study. Chapter 4 covers the system analysis and design, including architecture, workflow, and diagrams.

Chapter 5 describes the technology stack used in the project, such as React, HTML, CSS, JavaScript, Node.js, and Visual Studio Code. Chapter 6 explains the development and implementation process, including project structure, components, state management, filters, theme support, and responsive design. Chapter 7 presents the algorithms and logic used in the application.

Chapter 8 focuses on the user interface design and the visual structure of the system. Chapter 9 explains the source code modules and their working. Chapter 10 covers testing and validation, while Chapter 11 presents the results and output screens. Chapter 12 discusses future enhancements that can improve the system further. Finally, Chapter 13 concludes the report by summarizing the project achievements and learning outcomes.

This structured organization ensures that the report flows logically from introduction to implementation, testing, and conclusion. It also helps readers understand both the conceptual and practical aspects of the project in a clear and systematic way.

CHAPTER 2: LITERATURE REVIEW

2.1 Traditional Timetable Management Methods

Timetable management has always been an important part of academic administration in schools, colleges, and universities. Before the introduction of digital systems, timetables were prepared manually by academic staff using registers, charts, spreadsheets, or handwritten documents. These timetables were usually displayed on notice boards, circulated as printed sheets, or shared in departmental offices. Although this method served the basic purpose of informing students and faculty about class schedules, it had several practical limitations.

One of the major problems with traditional timetable management was the high possibility of human error. Since the data was entered manually, mistakes in subject names, room numbers, faculty assignments, or class timings could easily occur. Once printed, such mistakes were difficult to correct without reprinting the entire timetable. This made the process time-consuming and inefficient. In addition, whenever there was a change in schedule due to faculty absence, examination duty, or classroom unavailability, the timetable had to be updated manually and redistributed to all concerned users.

Another issue with traditional methods was poor accessibility. Students often had to physically visit notice boards or departmental offices to check their schedules. This was inconvenient, especially in large institutions where different departments maintained separate timetables. If a student belonged to multiple classes or elective groups, it became even harder to track the correct schedule. Faculty members also faced difficulty when they had to coordinate with several classes, practical sessions, or lab schedules across different days.

Traditional timetable systems also lacked flexibility. They were usually static and could not provide dynamic features such as subject-wise search, faculty filtering, classroom lookup, or personalized daily views. As educational institutions grew larger and academic schedules became more complex, the need for a more efficient and adaptable timetable management method became evident. The project report of the current system shows that the proposed application directly addresses these shortcomings by offering weekly and daily views, search functionality, subject filtering, faculty and classroom lists, and responsive interface support.

2.2 Existing Online Timetable Systems

With the development of the internet and web technologies, many institutions moved from manual timetable handling to online timetable systems. These systems were designed to display schedules digitally, often through institutional websites, dashboards, or intranet portals. The main purpose of these systems was to provide users with a centralized place where timetable information could be accessed quickly and updated more easily.

Existing online timetable systems generally offer features such as class schedule display, faculty allocation, lecture timing, and room information. Some systems are integrated with student portals or learning management systems, allowing users to view their class timings alongside notices, assignments, and attendance details. In more advanced setups, timetable information can also be linked to calendars, alerts, and mobile notifications.

However, many of these existing solutions are limited in design and usability. Some systems provide timetable data in a plain table format without interactive filtering or better visual grouping. Others are built using outdated interfaces that are not mobile-friendly or visually engaging. In several cases, the online timetable is simply a digital replacement of a printed chart, rather than a truly interactive scheduling tool.

Modern web-based applications, especially those built with front-end frameworks such as React, offer better opportunities for improving user experience. They can dynamically render timetable data, respond to user interactions instantly, and support features such as day navigation, search history, theme switching, and responsive layouts. The current project follows this modern approach by offering a dashboard-style timetable viewer with weekly and daily modes, filter options, and a polished interface. The report shows that the system is designed not only to display schedules but also to make timetable exploration easier and more intuitive.

2.3 Comparison of Existing Solutions

A comparison of traditional timetable methods and existing online timetable systems reveals a clear evolution in the way academic scheduling is handled. Traditional systems depend heavily on manual preparation, printed distribution, and physical notice boards. They are simple to create but difficult to update and maintain. Online systems, in contrast, reduce manual effort and improve accessibility by making timetable data available through web platforms.

In terms of accessibility, traditional systems score lower because users must physically look for the timetable or depend on printed copies. Online systems are better because they can be accessed from any device with internet connectivity. This is especially useful for students and faculty who need to check schedules on the move. The current project goes beyond basic accessibility by supporting responsive design, which means the timetable can be viewed comfortably on mobile phones, tablets, and desktops.

In terms of usability, traditional systems are limited to static display formats. Users cannot search for a specific subject or filter schedules by faculty name, room, or day. Many existing online systems improve this by offering searchable tables or digital dashboards, but not all of

them provide a smooth and visually organized experience. The present project improves usability by including subject-wise filtering, faculty filtering, classroom filtering, and navigation controls that help users move quickly between days and time slots.

.

When it comes to visual design, older systems are usually plain and text-heavy. They may convey information correctly but do not focus on readability or user experience. In contrast, modern web-based systems can use color-coded categories, attractive layouts, and theme customization to make the timetable more engaging. The project report mentions color-coded subjects, dark/light theme support, and premium glassmorphism styling, which are important improvements over conventional timetable systems.

.

Another important comparison is update handling. In traditional systems, any change requires manual correction and reprinting. In many older online systems, updates may still require page reloads or administrative intervention. Modern React-based systems are more flexible because they can update the displayed information dynamically through component re-rendering and state changes. This makes them more suitable for environments where timetable changes are frequent.

Overall, existing solutions have improved timetable management significantly, but there is still room for innovation. A truly effective timetable portal should combine accessibility, speed, visual clarity, responsiveness, and filtering capabilities. This is exactly the direction taken by the proposed project.

2.4 Limitations of Current Systems

Despite the availability of digital timetable systems, many current solutions still have several limitations. One major limitation is the lack of interactivity. A large number of timetable systems only display data in a basic table without allowing users to filter, search, or switch between different views. This makes it harder for users to find relevant information quickly, especially in institutions with multiple classes, subjects, and faculty members.

Another limitation is poor responsiveness. Many timetable websites are designed mainly for desktop screens and do not adapt well to smaller devices. Since students often use smartphones to access academic information, a non-responsive timetable system becomes inconvenient and less effective. A modern timetable portal should maintain readability and accessibility across all screen sizes. The current project addresses this through responsive design and flexible layout behavior.

.

A further limitation of many systems is the absence of advanced filtering. Users often need to locate a timetable by subject, teacher, or room, but older systems may not support these functions. Without filtering, users must manually scan the entire timetable, which increases effort and reduces efficiency. The proposed timetable viewer improves this by including subject-wise filtering, faculty directory support, and classroom listing features.

.

Limited personalization is another issue. Some systems present the same static information to every user without allowing theme changes, preference-based layouts, or alternate views. Modern users expect interfaces that are comfortable and adaptable. Features such as dark mode, custom color themes, and dashboard widgets improve usability and reduce visual strain. These elements are included in the proposed project, showing an attempt to move beyond standard timetable display behavior.

.

Many systems also lack additional academic utilities. A timetable portal can be more valuable if it includes attendance tracking, class progress indicators, notification support, and export options. However, several existing solutions stop at schedule display and do not integrate these practical features. The current project includes attendance tracker elements, print/export support, dashboard summaries, and offline/PWA-oriented functionality, making it more comprehensive than a basic timetable viewer.

.

Finally, maintenance and scalability are common problems. Some timetable systems are difficult to update when class structures change or when new departments are added. If the system is not modular, then small changes can require major code modifications. In contrast, a component-based React application can be more scalable because sections such as daily view, weekly view, filters, and faculty cards can be developed and maintained separately.

2.5 Need for Modern Web-Based Timetable Portals

The limitations of traditional and older online systems clearly show the need for modern web-based timetable portals. Academic institutions now operate in a fast-paced environment where information must be updated and shared quickly. Students need instant access to class schedules, faculty details, room assignments, and timetable changes. A web-based portal provides the best solution because it combines accessibility, flexibility, and real-time user interaction.

Modern web portals are especially important because they support centralized information management. Instead of relying on paper notices or scattered communication channels, all timetable data can be stored in one organized system. This helps ensure consistency and reduces the risk of outdated information being circulated. It also improves coordination among students, faculty, and administrative staff.

Another major reason for adopting modern portals is user experience. Current users expect clean interfaces, easy navigation, search tools, and responsive layouts. A timetable system must therefore do more than simply show time slots; it must present information in a way that is easy to understand and use. The proposed project reflects this need by providing weekly and daily views, advanced filters, dashboard elements, and a visually polished interface.

Modern web-based portals also support better scalability. As a college grows, more courses, classes, and faculty members can be added without redesigning the whole system. If the application is built using component-based architecture, the timetable can be expanded or modified with less effort. This is one of the advantages of using React for the front end. It allows the interface to update dynamically and supports reusable components for timetable grids, cards, filters, and summaries.

In addition, modern portals can integrate features that improve productivity and planning. These may include calendar synchronization, notifications, offline access, print-friendly output, and attendance management. The project report already includes several such enhancements, including PDF/print export, dark/light themes, attendance tracking, and PWA support. These features show that the system is designed not only as a timetable viewer but also as a broader academic support tool.

The need for modern timetable portals is also connected to digital transformation in education. Institutions are increasingly adopting online tools to improve efficiency and student engagement. A well-designed timetable portal contributes to this transformation by reducing confusion, saving time, and improving access to schedule information. It becomes a practical digital service that supports everyday academic life.

CHAPTER 3: REQUIREMENT ANALYSIS

Requirement analysis is one of the most important stages in software development because it helps define what the system must do, what resources are needed, and whether the proposed solution is practical. In the case of the **College Timetable Viewer using React, HTML, and CSS**, requirement analysis ensures that the application is designed to solve the real problems faced by students and faculty members while remaining simple, fast, and user-friendly. The project aims to provide a centralized and responsive timetable viewer that displays schedules in a clear and organized format, and this directly shapes the requirements of the system.

The report shows that the application includes weekly and daily timetable views, subject-wise filtering, faculty filtering, classroom filtering, day navigation, search functionality, responsive design, theme support, attendance tracking, PWA support, and PDF/print export. These features define not only the functional scope of the system but also its technical and usability requirements.

3.1 Functional Requirements

Functional requirements describe the actual operations and services that the system must provide to users. In this project, the timetable viewer is expected to perform several key functions that make schedule access simple and efficient.

The first functional requirement is the ability to display timetable data in a structured format. The system must show college timetable information clearly so that users can view schedules without confusion. The report mentions both **weekly timetable view** and **daily timetable view**, which means the system should allow users to switch between a complete week overview and a focused day-by-day schedule. This is useful for users who want either a broad or detailed view of class timings.

The second functional requirement is **subject-wise filtering**. Users should be able to search timetable entries by subject name and quickly locate the required class. The project report specifically includes filtering by subject, faculty, and classroom, showing that the system is intended to support multiple search modes for different users.

The third functional requirement is **faculty information display**. Faculty members are an important part of the timetable system because students often need to know who is teaching a particular subject, the office hours, and availability. The application therefore includes a faculty

directory with role, contact, and office-hour details. This improves communication and makes the timetable more informative than a simple time table table.

The fourth functional requirement is **classroom and laboratory information**. The system should display room numbers, building names, seating capacity, and equipment details where needed. This is especially important for laboratory sessions and practical classes where room allocation matters. The report includes classroom filtering and laboratory listing, which confirms that the timetable viewer is designed to support both academic and infrastructural information.

.

The fifth functional requirement is **navigation support**. Users must be able to move between days and timetables easily through next/previous buttons or quick day selectors. This helps students and faculty access information faster and reduces the effort needed to locate a specific lecture or session. The report describes day navigation as part of the implemented features.

.

The sixth functional requirement is **search functionality**. The system should allow users to search timetable entries efficiently, ideally with autocomplete suggestions or recall history. This feature is useful when the timetable contains many subjects, labs, and faculty names. The project report includes advanced search with autocomplete suggestions and history recall logs, which makes the application more practical and user-oriented.

.

The seventh functional requirement is **theme management**. Users should be able to switch between dark and light modes, along with custom themes such as Violet, Emerald, Cyberpunk, and Amber. This improves visual comfort and gives users a personalized experience. The report shows that theme support is already included in the system.

The eighth functional requirement is **attendance tracking**. The application should support attendance increments, status indicators, and goal calculation to predict how many classes are needed to achieve a target attendance percentage. The report mentions a goal calculator for reaching a 75% target, which indicates that the system is not only a timetable viewer but also a useful academic support tool.

.

The ninth functional requirement is **offline and print support**. The timetable viewer should be able to work as a progressive web app and also generate print-friendly timetable sheets. The project report includes PWA support through service worker caching and PDF/print export using media print styles. This ensures that users can access the timetable even in limited connectivity and also keep a physical copy when needed.

3.2 Non-Functional Requirements

Non-functional requirements describe the quality attributes of the system rather than specific actions. These requirements ensure that the application is fast, reliable, easy to use, and suitable for real-world conditions.

One of the most important non-functional requirements is **usability**. The system must be easy to understand and navigate even for first-time users. Since the project is meant for students and faculty, the interface should avoid unnecessary complexity and present timetable data in a clean, organized layout. The report highlights an intuitive interface and modern user interface design, showing that usability was an important part of the project.

Another key non-functional requirement is **responsiveness**. The application should work smoothly on different screen sizes, including desktop computers, laptops, tablets, and mobile phones. The report mentions a mobile-friendly layout using CSS Flexbox and media queries, which means the design is expected to adapt properly to different devices.

.

Performance is also essential. The application should load timetable data quickly and respond instantly when users apply filters, switch views, or change themes. Since the system uses React components and state management, it should be able to update portions of the interface efficiently without full page reloads. The logic described in the report confirms that the timetable is rendered dynamically and filtered on-the-fly.

Reliability is another non-functional requirement. The system should display accurate timetable data without breaking due to minor updates or user interactions. Since the report stores timetable information in centralized structured objects, data consistency becomes easier to manage. The application should also behave predictably when users navigate through days, subjects, or faculty entries.

.

Maintainability is important because the timetable structure may change over time. New subjects, faculty members, classrooms, or semesters may be added in the future. Therefore, the code should be modular and easy to update. A React-based component structure helps achieve this because individual parts of the interface can be modified without rebuilding the whole system. The project report indicates that data, filtering, and UI rendering are organized through separate logic steps, which supports maintainability.

.

Security is a basic requirement, even in a front-end project. The system should avoid exposing sensitive data and should handle user input safely where search or filter inputs are used. Although this project is primarily a front-end timetable viewer, safe handling of inputs and clean data rendering remain important considerations.

Finally, **compatibility** must be ensured. The application should run correctly in modern browsers such as Chrome, Edge, and Firefox, as mentioned in the report. Cross-browser support is important so that students and faculty can access the timetable without technical issues.

.

3.3 Software Requirements

Software requirements refer to the applications, frameworks, tools, and environments needed to develop and run the system. According to the project report, the required software includes **React.js, HTML5, CSS3, Visual Studio Code, Node.js, and a modern web browser**

such as **Chrome, Edge, or Firefox**. These tools form the complete development and execution environment for the timetable viewer.

.

React.js is the main front-end framework used in the project. It is suitable for building interactive user interfaces and handling dynamic data rendering. Since the timetable viewer needs daily and weekly views, filter actions, and theme switching, React provides the necessary structure for state-driven updates.

HTML5 is used to structure the content of the web page. It defines the layout elements, headings, sections, cards, and form controls required for displaying timetable details.

CSS3 is used for styling the interface. The report mentions responsive design, dark theme support, glassmorphism effects, and print-media overrides, all of which depend on CSS3 styling capabilities.

Visual Studio Code is used as the code editor because it provides a convenient environment for writing and managing project files. **Node.js** is required to support the React development environment and package management. A modern **web browser** is necessary to run and test the application.

The report also indicates the use of **service worker offline caching** and **manifest settings** for PWA support, which are software-level features that improve accessibility and installation behavior. In addition, the project uses separate source files such as App.jsx and styles.css, showing that the application follows a modular front-end structure.

.

3.4 Hardware Requirements

Hardware requirements define the minimum physical resources needed to develop and run the project smoothly. Since this is a web-based timetable viewer, the hardware requirements are not very heavy, but they still need to be sufficient for running React development tools and a browser environment.

A basic development machine should include a **modern processor**, **minimum 4 GB RAM**, and **adequate storage** for source files and project dependencies. However, for smoother development, **8 GB RAM or more** is recommended, especially if the user is running VS Code, browser tabs, and Node-based development servers simultaneously.

The system should also have a **display device** with reasonable screen resolution to test responsive layouts properly. Since the application is designed to work on various screen sizes, testing on both desktop and mobile emulation is important.

For storage, only a small amount is required because the project is primarily front-end based. A **stable internet connection** is helpful during development for installing dependencies, but the final application itself is designed with offline support through PWA features. The report's mention of offline caching shows that the system is intended to remain usable even when internet access is limited.

3.5 User Requirements

User requirements describe what the intended users expect from the system. In this project, the users are mainly **students, faculty members, and potentially administrators**. Each group has different needs, but all of them require quick and clear access to timetable information.

Students need an easy way to view their class schedules, check subject timings, locate classrooms, and understand who is teaching each class. They also benefit from daily and weekly views, because these help them plan their study time and attendance better. The project report directly states that the system is designed to help students and faculty members view college timetables efficiently.

Faculty members need timetable access for lecture planning, class coordination, and room management. They may also need to check other faculty schedules, office hours, and availability. The faculty filtering and directory functions in the project address this need.

Administrators or timetable coordinators need a system that is organized, easy to update, and capable of handling changes in class schedules. They also need supporting features such as classroom lists, category colors, and print/export options to manage academic planning effectively. Since the system includes centralized timetable data and a structured rendering logic, it satisfies these user expectations well.

.

Users also expect the interface to be fast, attractive, and mobile-friendly. The report's responsive design and modern UI features show that the project is aligned with these expectations. Theme switching, search, and filter options make the system more convenient for everyday use.

3.6 Feasibility Study

Feasibility study is conducted to determine whether the proposed system is practical from technical, economic, and operational perspectives. For the **College Timetable Viewer**, the feasibility study shows that the system is realistic and well-suited for implementation using modern web technologies.

Technical Feasibility

The project is technically feasible because it is built using widely supported technologies such as React, HTML5, CSS3, Node.js, and a standard web browser. These technologies are stable, well-documented, and suitable for building interactive front-end applications. The project report confirms that the system successfully demonstrates React components, state management, filtering, and responsive web design. This means the required technical skills and tools are available and appropriate for the project.

The use of centralized timetable objects, dynamic rendering, filter logic, and theme switching also confirms that the system can be implemented without major technical barriers. Since the project is front-end focused, it does not require complex backend infrastructure for its core functionality. This makes development easier and more manageable.

Economic Feasibility

The project is economically feasible because it uses open-source and low-cost software tools. React, HTML, CSS, and Node.js are free to use, and Visual Studio Code is also available without

cost. A standard computer and browser are enough to build and run the application. Therefore, the cost of development is low compared to a large-scale enterprise system.

Another economic advantage is that the system can reduce manual work in timetable management. Instead of printing and reprinting timetables or spending extra time distributing schedule updates, the digital system provides quick access and easier maintenance. This saves both time and effort for the institution. The inclusion of print/export support and PWA features also adds value without requiring expensive infrastructure.

Operational Feasibility

The project is operationally feasible because it is easy for students, faculty, and administrators to understand and use. The interface is designed to be intuitive, responsive, and visually clear. Users can view weekly and daily timetables, search subjects, filter faculty and classrooms, and switch themes without training or special instructions. The report's emphasis on a user-friendly web application confirms that operational ease is a key goal of the system.

.

The system also supports practical academic usage because it provides information in a format that users already expect. Since timetable checking is a regular daily activity, a well-designed digital portal can be quickly adopted by the institution. Features like attendance tracking, dashboard overview, and offline access further improve its usefulness in real-world operation.

CHAPTER 4: SYSTEM ANALYSIS AND DESIGN

System Analysis and Design is one of the most important phases in software development. It defines how the proposed system will function, how different components interact with each other, and how users will access and process information. For the **College Timetable Viewer using React, HTML, and CSS**, system analysis focuses on understanding the flow of timetable data, user interactions, filtering mechanisms, navigation controls, attendance tracking, and interface design. The project is designed to provide a centralized, responsive, and user-friendly platform where students and faculty members can efficiently view and manage timetable information. The report indicates that the system supports weekly and daily timetable views, subject and faculty filtering, classroom directories, attendance tracking, dashboard widgets, theme management, and print/export functionality.

.

The design of the system follows a component-based architecture using React.js, which allows different modules to operate independently while maintaining overall system consistency. This chapter explains the architecture, workflow, diagrams, and design models used in the development of the timetable viewer.

4.1 Proposed System Architecture

The proposed architecture of the College Timetable Viewer follows a modern front-end web application structure. The system is primarily divided into three layers:

1. Presentation Layer

The presentation layer consists of the user interface developed using HTML, CSS, and React components. This layer is responsible for displaying timetable information, navigation controls, dashboards, filters, attendance data, and theme settings.

Users interact directly with this layer through web browsers. Features such as daily timetable view, weekly timetable view, search boxes, subject filters, and classroom directories are presented here.

2. Application Logic Layer

The application logic layer manages all processing operations. This layer is implemented through React state management and JavaScript functions.

Major responsibilities include:

- Timetable rendering
- Search processing
- Subject filtering
- Faculty filtering
- Attendance calculations
- Theme switching
- Navigation controls
- Dashboard updates

The report states that timetable information is dynamically rendered using React components and state-based updates. This layer ensures that changes are reflected instantly without requiring page reloads.

.

3. Data Layer

The data layer stores timetable information in structured JavaScript objects.

Data includes:

- Subject Name
- Faculty Name
- Room Number
- Lecture Time
- Subject Category
- Attendance Records
- Theme Settings

The report specifies that timetable information is stored in centralized objects inside `timetableData.js` and accessed dynamically by React components.

Architecture Flow

User



React User Interface



Application Logic



Timetable Data Objects



Updated User Interface

This architecture ensures simplicity, maintainability, and scalability.

4.2 System Workflow

The workflow describes how users interact with the system and how information moves through different modules.

The workflow begins when a user opens the timetable viewer in a web browser. The application loads timetable data from predefined data structures and renders the dashboard screen.

The user may perform several actions:

- View weekly timetable
- View daily timetable
- Search subjects
- Filter faculty
- Filter classrooms
- Change themes
- Update attendance
- Export timetable

When a user enters a search term, React processes the input and filters timetable entries based on matching keywords. Similarly, when a user selects a faculty member or classroom, only relevant timetable entries are displayed.

If attendance is updated, the attendance module recalculates percentages and goal predictions instantly. Theme changes are applied by modifying root CSS variables and updating state values.

The workflow ends when the user exits the application.

Workflow Steps

1. User opens application.

2. Timetable data loads.
3. Dashboard screen appears.
4. User selects operation.
5. System processes request.
6. Updated timetable is displayed.
7. User continues interaction or exits.

This workflow provides smooth navigation and responsive interaction throughout the application.

.

4.3 Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) represents the movement of information within the system.

Level 0 DFD (Context Diagram)

The Level 0 DFD shows the entire timetable system as a single process.

External Entities

- Student
- Faculty Member

Main Process

- College Timetable Viewer

Data Flows

Student → System

- View Schedule Request
- Search Request
- Attendance Request

System → Student

- Timetable Information
- Attendance Status
- Filtered Results

Faculty → System

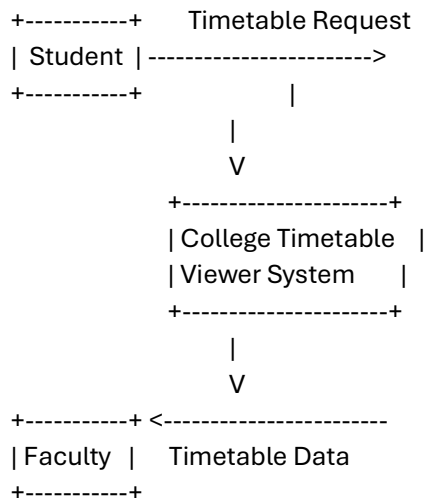
- Faculty Schedule Request

- Room Information Request

System → Faculty

- Faculty Timetable
- Classroom Information

Level 0 DFD Representation



Level 1 DFD

Level 1 DFD breaks the system into smaller processes.

Processes

- P1 – Load Timetable Data
- P2 – Search & Filter Data
- P3 – Attendance Management
- P4 – Theme Management
- P5 – Export & Print Services

Data Store

- D1 – Timetable Database

Flow

- Student
 - ↓
 - P1 Load Timetable
 - ↓
 - D1 Timetable Data
- P2 Search & Filter

↓

Filtered Timetable

P3 Attendance Module

↓

Attendance Results

P4 Theme Module

↓

Theme Settings

P5 Export Module

↓

PDF / Print Output

This decomposition helps developers understand internal system functionality.

4.4 Use Case Diagram

A Use Case Diagram represents interactions between users and the system.

Actors

1. Student
2. Faculty Member

Use Cases

- View Weekly Timetable
- View Daily Timetable
- Search Subject
- Filter Faculty
- Filter Classroom
- Check Attendance
- Calculate Attendance Goal
- Change Theme
- Export Timetable
- Print Timetable

Use Case Description

Students primarily interact with timetable viewing, attendance monitoring, and filtering functions.

Faculty members use timetable viewing, faculty filtering, and classroom information features.

Text Representation

Student

- |— View Weekly Timetable
- |— View Daily Timetable
- |— Search Subject
- |— Check Attendance
- |— Change Theme
- |— Export Timetable

Faculty

- |— View Timetable
- |— Search Subject
- |— Filter Classroom
- |— View Faculty Schedule

4.5 Activity Diagram

The Activity Diagram describes the sequence of actions performed by the user.

Activity Flow

Start

↓

Open Application

↓

Load Timetable Data

↓

Display Dashboard

↓

Select Option

↓

[View Timetable?]

↓

Display Timetable

↓

[Apply Filter?]

↓

Filter Results

↓

Display Output

↓

End

The activity diagram highlights how users navigate through the application and receive updated information.

4.6 Sequence Diagram

The Sequence Diagram shows interactions between the user, interface, logic layer, and data source over time.

Objects

- User
- React Interface
- Application Logic
- Timetable Data

Sequence Flow

User

↓

React Interface

↓

Application Logic

↓

Timetable Data

Interaction Sequence

1. User requests timetable.
2. Interface sends request.
3. Logic retrieves data.
4. Data returned.
5. Interface displays timetable.
6. User applies filter.
7. Logic processes filter.
8. Updated timetable displayed.

This sequence ensures efficient communication among system components.

4.7 Class Diagram

The Class Diagram represents the structure of classes and their relationships.

Class 1: Timetable

Attributes:

- day
- time

- subject
- faculty
- classroom

Methods:

- getTimetable()
 - updateTimetable()
-

Class 2: Faculty

Attributes:

- facultyId
- facultyName
- department

Methods:

- getFacultyInfo()
-

Class 3: Classroom

Attributes:

- roomId
- roomName
- capacity

Methods:

- getRoomInfo()
-

Class 4: Attendance

Attributes:

- attendedClasses
- totalClasses

Methods:

- calculatePercentage()
 - predictGoal()
-

Relationship Diagram

Timetable

|

| uses

|

Faculty

Timetable

|

| assigned to

|

Classroom

Student

|

| monitors

|

Attendance

The class diagram improves understanding of object-oriented relationships used in the project.

4.8 Component Diagram

The Component Diagram shows the major software modules of the application.

Components

1. Dashboard Component
2. Weekly Timetable Component
3. Daily Timetable Component
4. Search Component
5. Faculty Directory Component
6. Classroom Directory Component
7. Attendance Component
8. Theme Manager Component
9. Export Component

App.jsx

|

└─ Dashboard

└─ Weekly Timetable

└─ Daily Timetable

- └─ Search Module
- └─ Faculty Directory
- └─ Classroom Directory
- └─ Attendance Tracker
- └─ Theme Manager
- └─ Export Service

CHAPTER 5: TECHNOLOGY STACK

The success of any software application largely depends on the technologies used during its development. Selecting the appropriate technology stack ensures better performance, maintainability, scalability, and user experience. The **College Timetable Viewer using React, HTML, and CSS** has been developed using modern web technologies that support responsive design, dynamic content rendering, efficient state management, and cross-platform accessibility. The project utilizes React.js for building the user interface, HTML5 for page structure, CSS3 for styling, JavaScript ES6 for programming logic, Node.js for development support, Visual Studio Code as the development environment, and modern web browsers for execution and testing. These technologies collectively provide a strong foundation for building a fast, interactive, and user-friendly timetable management application. The report identifies React.js, HTML5, CSS3, Node.js, Visual Studio Code, and web browsers as the primary software requirements used in the project.

.

5.1 Introduction to React.js

React.js is a popular open-source JavaScript library developed by Meta Platforms for building interactive user interfaces, especially for single-page applications (SPAs). It follows a component-based architecture that allows developers to divide the user interface into reusable and independent components. This approach improves code organization, maintainability, and scalability.

In the College Timetable Viewer project, React.js serves as the core technology responsible for rendering timetable information dynamically. Instead of reloading the entire page whenever data changes, React updates only the necessary components through its Virtual DOM mechanism. This significantly improves application performance and user experience.

One of the major advantages of React.js is its state management capability. The project uses React state variables to manage selected days, active timetable views, search filters, attendance records, and theme settings. Whenever a user changes a filter or selects a different timetable view, React automatically updates the displayed information without requiring a page refresh. According to the project report, timetable information is dynamically rendered using React components and state routers, ensuring smooth transitions and real-time updates.

React also supports component reusability. For example, timetable cards, navigation buttons, dashboard widgets, faculty information panels, and attendance modules can be created as separate reusable components. This reduces code duplication and simplifies future modifications.

Another important feature of React is declarative programming. Developers specify what the user interface should look like based on the current application state, and React handles the necessary updates automatically. This makes application logic easier to understand and maintain.

The popularity of React in modern web development is another reason for selecting it in this project. It has strong community support, extensive documentation, and compatibility with many third-party libraries. As a result, React provides a reliable and future-proof foundation for developing academic management applications such as the College Timetable Viewer.

5.2 HTML5 Fundamentals

HTML5 (HyperText Markup Language Version 5) is the standard markup language used for creating and structuring web pages. It provides the basic framework upon which all website content is displayed. In this project, HTML5 is responsible for defining the structure of timetable screens, dashboards, navigation menus, search forms, attendance panels, and other user interface elements.

HTML5 introduced several improvements over previous versions of HTML, including semantic tags, multimedia support, improved forms, and better accessibility. Semantic elements such as `<header>`, `<section>`, `<article>`, `<nav>`, and `<footer>` make the structure of web pages more meaningful and easier to understand for both developers and search engines.

The College Timetable Viewer relies on HTML5 to organize timetable information into structured layouts. Timetable grids, daily schedule cards, faculty information panels, classroom lists, and dashboard sections are all built using HTML elements. The proper use of semantic tags improves readability, maintainability, and accessibility.

Another significant advantage of HTML5 is its support for modern web applications. Features such as local storage, offline capabilities, and multimedia integration contribute to improved user experiences. Since the project includes Progressive Web App (PWA) support and offline caching, HTML5 features play an important role in enabling these functionalities.

HTML5 also ensures compatibility across different browsers and devices. Whether users access the timetable through desktops, laptops, tablets, or smartphones, the HTML structure remains consistent and reliable.

In addition, HTML5 supports integration with CSS3 and JavaScript, allowing developers to create visually appealing and highly interactive interfaces. This combination forms the backbone of modern web development and serves as the foundation of the proposed timetable management system.

5.3 CSS3 Features

CSS3 (Cascading Style Sheets Version 3) is the styling language used to control the visual appearance of web pages. While HTML defines structure, CSS3 determines colors, layouts, typography, spacing, animations, and responsiveness. In the College Timetable Viewer project, CSS3 is responsible for creating an attractive, modern, and user-friendly interface.

The project report highlights several advanced CSS3 features, including responsive design, glassmorphism effects, dark/light themes, custom color themes, animations, and print-friendly layouts. These features significantly enhance the visual quality and usability of the application.

.

One of the most important CSS3 features used in this project is **Responsive Web Design**. Through Flexbox, Grid Layouts, and Media Queries, the interface automatically adjusts to different screen sizes. This ensures that timetable information remains readable and accessible on smartphones, tablets, laptops, and desktop computers.

CSS3 also supports custom properties, commonly known as CSS variables. These variables allow dynamic theme switching by changing color schemes at runtime. The project uses this capability to implement Dark Mode, Light Mode, Violet Theme, Emerald Theme, Cyberpunk Theme, and Amber Theme. Theme changes occur instantly without reloading the page.

.

Another important feature is animation support. CSS3 animations and transitions create smooth effects when users navigate between timetable views or apply filters. These visual enhancements improve user engagement and create a more professional appearance.

Glassmorphism styling is another notable design feature mentioned in the project report. This design approach uses transparency, blur effects, and layered shadows to create a modern premium interface. Combined with responsive layouts and color-coded timetable categories, it contributes to an intuitive and visually appealing user experience.

CSS3 also includes print media queries, which are used in the project's PDF and print export functionality. These rules optimize timetable layouts specifically for printing, ensuring that exported schedules remain organized and readable.

.

5.4 JavaScript ES6 Features

JavaScript is the primary programming language used for web application development. ES6 (ECMAScript 2015) introduced several powerful features that simplify coding and improve performance. The College Timetable Viewer utilizes JavaScript ES6 extensively for handling timetable data, implementing filtering logic, managing application state, and responding to user interactions.

One of the most important ES6 features used in the project is **Arrow Functions**. Arrow functions provide a shorter syntax for writing functions and improve code readability. They are commonly used in React components for handling events and processing timetable data.

Another useful feature is **Template Literals**, which allow developers to create dynamic strings more efficiently. These are particularly helpful when generating timetable entries or displaying user-specific information.

Destructuring Assignment simplifies the extraction of values from objects and arrays. Since timetable information is stored in structured JavaScript objects, destructuring improves code clarity and reduces repetition.

Modules and Imports are also widely used. React applications are organized into multiple files and components, making import/export functionality essential for managing project structure. The project report references files such as App.jsx and timetableData.js, which rely on modular programming practices.

.

JavaScript ES6 also introduced **let** and **const** keywords, which provide better variable scope control compared to traditional var declarations. These improve code safety and maintainability.

The project uses JavaScript filtering methods to dynamically search timetable entries based on subjects, faculty members, classrooms, and user-defined queries. The report specifically describes filtering operations using JavaScript functions that match timetable data against search inputs.

.

Overall, JavaScript ES6 provides the logic and interactivity necessary to transform a static timetable into a dynamic and responsive web application.

5.5 Node.js Environment

Node.js is a JavaScript runtime environment that allows developers to execute JavaScript code outside web browsers. It plays an important role in React development because it provides package management, build tools, and development servers.

In the College Timetable Viewer project, Node.js is used primarily during the development phase. It enables the installation of React dependencies, manages project packages through npm (Node Package Manager), and provides tools for compiling and serving the application locally.

Node.js offers several advantages:

- Fast execution using the V8 JavaScript engine.
- Efficient package management through npm.
- Large ecosystem of libraries and tools.
- Support for modern development workflows.

React applications typically rely on Node.js to manage project dependencies such as React libraries, routing modules, and development utilities. Without Node.js, setting up and maintaining a React project would be significantly more difficult.

Another advantage of Node.js is its compatibility with Progressive Web App development. Since the project includes PWA support through service workers and manifest files, Node.js assists in managing these configurations during development and deployment.

.

Node.js therefore serves as the backbone of the development environment, enabling efficient coding, testing, and deployment of the timetable viewer.

5.6 Visual Studio Code IDE

Visual Studio Code (VS Code) is a lightweight yet powerful source code editor developed by Microsoft. It is one of the most popular development environments for web applications due to its speed, flexibility, and extensive extension ecosystem.

The College Timetable Viewer project was developed using Visual Studio Code because it provides a productive environment for writing React, HTML, CSS, and JavaScript code. Its integrated terminal allows developers to run Node.js commands and React development servers without leaving the editor.

VS Code offers several important features:

- Syntax highlighting
- Code auto-completion
- Error detection
- Integrated debugging
- Git integration
- Extension marketplace

These features improve development efficiency and reduce coding errors. For React projects, VS Code provides extensions that simplify JSX development, component management, and formatting.

Another advantage is its support for live previews and debugging tools. Developers can test timetable layouts, inspect UI elements, and identify issues quickly. This is particularly useful for responsive design testing and theme customization.

Because of its user-friendly interface and extensive capabilities, Visual Studio Code is an ideal development environment for modern web applications such as the College Timetable Viewer.

5.7 Browser Compatibility

Browser compatibility refers to the ability of a web application to function correctly across different web browsers and devices. Ensuring compatibility is essential because users may access the timetable viewer through various browsers depending on their preferences and operating systems.

According to the project report, the application is designed to support modern browsers such as Google Chrome, Microsoft Edge, and Mozilla Firefox. These browsers provide full support for React applications, HTML5 features, CSS3 styling, and JavaScript ES6 functionality.

.

Cross-browser compatibility is achieved through standards-compliant coding practices. HTML5 ensures consistent document structure, CSS3 provides responsive layouts, and React handles dynamic rendering in a browser-independent manner.

Responsive design further enhances compatibility by allowing the application to adapt to different screen resolutions. Whether accessed from a desktop computer, laptop, tablet, or smartphone, the timetable viewer maintains functionality and readability.

Browser compatibility testing is important because different browsers may render styles and scripts differently. During development, testing should be performed on multiple browsers to ensure consistent appearance and behavior. Features such as theme switching, attendance tracking, timetable filtering, and PDF export should function correctly regardless of the browser being used.

The project's mobile-friendly design, responsive layout, and standards-based implementation ensure that users receive a consistent experience across platforms. This contributes significantly to the overall reliability and usability of the system.

CHAPTER 6: SYSTEM DEVELOPMENT AND IMPLEMENTATION

The development of the **College Timetable Viewer using React, HTML, and CSS** was carried out to create a modern, responsive, and easy-to-use timetable management portal for students and faculty members. The system was built to solve the common problems of traditional timetable handling, such as lack of accessibility, poor organization, difficult updates, and limited search or filtering options. The project includes weekly and daily timetable views, subject-wise filtering, faculty and classroom directories, day navigation, search functionality, dark/light themes, attendance tracking, PWA support, and PDF/print export features. These features guided the implementation process and shaped the internal structure of the application.

.

This chapter explains how the project was structured, how the source files were organized, how React components were designed, how state was managed, how search and filtering were implemented, and how other major features were developed. The project report clearly indicates that timetable data was stored in structured objects, React components dynamically rendered the timetable, filters worked on the dataset, and theme settings were switched through HTML attributes and CSS variables.

.

6.1 Project Structure Overview

The overall project structure follows a modular front-end architecture. Since the application is built with React, the code is divided into reusable files and components so that each part of the timetable system can be handled separately. This approach improves clarity, maintainability, and reusability. The main entry file, App.jsx, acts as the central component that coordinates the display of dashboards, timetable views, filters, and other user interactions. The report mentions App.jsx and styles.css as the key source files used in the implementation.

.

At a high level, the project includes the following functional sections:

- Dashboard overview
- Weekly timetable view
- Daily timetable view
- Subject filter module
- Faculty directory module
- Classroom directory module
- Attendance tracker
- Theme controller
- Export/print module

Each section is either rendered as a component or handled through logical state changes in React. This structure makes the application flexible and allows future enhancements to be added without rewriting the entire codebase. The result of the project confirms that the application successfully demonstrates React components, state management, filtering, and responsive web design.

.

6.2 Folder Organization

The folder organization of the project is kept simple and practical so that source files can be managed efficiently. A typical React-based project of this kind usually contains folders and files for components, data, styles, public assets, and configuration. In this timetable viewer, the main implementation is centered around the timetable data file and the React component file, along with a dedicated stylesheet. The report specifically mentions that timetable information is stored in timetableData.js, while the UI logic is handled through App.jsx and the styling is managed in styles.css.

.

A well-organized folder structure helps in the following ways:

- It separates data from presentation logic.
- It makes the code easier to debug and update.

- It supports component reuse.
- It improves readability for future developers.

The timetable data file contains structured records for subjects, faculty names, room numbers, timings, and colors. The React file handles rendering and user interaction, while the stylesheet manages layout, colors, spacing, animations, and responsive behavior. This separation of responsibilities is one of the strengths of the project design. The report also shows that the system is built around centralized structured objects, which further supports organized development.

6.3 React Components Design

React component design is the core of the application. React allows the interface to be divided into small and reusable units, each responsible for a specific part of the user interface. In the timetable viewer, components are used to display the dashboard, timetable grids, filters, faculty lists, classroom lists, attendance tracker, and theme controls. This component-based approach makes the system easier to develop and maintain.

The report explains that React components dynamically render timetable information using state routers and day-slot mapping. The timetable cells are generated by mapping over days and lecture slots, and current-day/current-lecture checks are performed using system clock comparisons.

.

Some likely component responsibilities include:

- **Header/Dashboard Component:** Displays the title, summary widgets, and notifications.
- **Weekly Timetable Component:** Shows the timetable in grid format from Monday to Saturday.
- **Daily Timetable Component:** Shows the selected day in chronological card style.
- **Search/Filter Component:** Accepts user input and updates the results.
- **Faculty Component:** Displays faculty role, contact, office hours, and availability.
- **Classroom Component:** Displays room details, capacity, building, and equipment.
- **Theme Component:** Controls switching between dark/light and custom themes.
- **Attendance Component:** Tracks attendance status and target progress.
- **Export Component:** Handles print-friendly output.

The use of separate components makes the interface more scalable. If a new feature is added later, it can be inserted as a new component without affecting the rest of the system. This is one of the most important reasons React was chosen for this project. The final report confirms that the application successfully demonstrates a modern timetable management interface using React.

6.4 State Management Implementation

State management is essential in this project because the timetable viewer depends on user interactions such as selecting a day, searching a subject, toggling themes, and updating attendance records. React state is used to store and update values dynamically. Whenever a user performs an action, the state changes and the relevant part of the UI is re-rendered automatically.

The report clearly states that configuration states such as active tabs, theme settings, favorites, and attendance counters are loaded from local storage and updated during runtime. It also mentions that buttons and select lists update state elements like `selectedDay` or active tabs, causing React to trigger smooth layout transitions.

.

Typical state variables in the application may include:

- Selected day
- Search query
- Filtered subject list
- Active theme
- Current timetable view
- Attendance values
- Export mode

State management improves the user experience because it removes the need for page reloads. For example, when a user switches from weekly view to daily view, the timetable updates instantly. Similarly, when a search term is entered, the displayed records change immediately based on the filter logic.

The use of state also helps preserve user preferences such as theme selection and last viewed schedule. Since the project stores some settings in local storage, the application can restore these preferences when reopened. This creates a more personalized and practical system.

6.5 Routing and Navigation

Routing and navigation in the timetable viewer are designed to make movement between different timetable sections smooth and intuitive. Although the project is mainly a single-page application, the navigation behavior is handled through component switching and state-based view changes rather than traditional page reloads. This is appropriate for a React-based dashboard-style system.

The report mentions day navigation through next/previous controls and quick day selectors, which allow users to move quickly across the timetable. It also notes that buttons and select lists update state elements, producing smooth layout transitions.

The navigation system likely includes:

- Home/Dashboard navigation
- Weekly timetable selection
- Daily timetable selection
- Subject filter access
- Faculty/classroom directory access
- Attendance section access
- Theme switching controls

This navigation style is user-friendly because users do not need to open separate pages for every function. Instead, the interface responds quickly to button clicks and selection changes. Day navigation is especially important because it lets students and faculty immediately check the timetable for a particular day without scanning the full week.

The use of React makes this navigation efficient because only the required component is updated. This keeps the application fast and visually smooth.

6.6 Timetable Data Storage Structure

Timetable data is one of the most important parts of the system. The report explains that timetable information such as subjects, faculty names, classrooms, timings, and category colors is stored in centralized structured objects inside `timetableData.js`. It even provides an example object showing a Monday slot with subject, lecture type, faculty, room, and color.

.

A structured timetable data format is useful because it allows the interface to display information dynamically rather than hard-coding timetable values into the UI. Each record can contain fields such as:

- Day
- Time slot
- Subject name
- Subject type
- Faculty name
- Room number
- Color code
- Category label

This approach has several advantages. First, it makes the timetable easier to update because changes can be made only in the data file. Second, it helps the filtering system because search

and filter logic can be applied directly to structured records. Third, it improves readability because each entry contains clearly defined properties.

The data structure also supports category-based color coding for lectures, labs, and seminars. This makes timetable visualization clearer and more attractive. Since the timetable is used repeatedly in weekly and daily views, storing the data in a structured and centralized way is essential for the system's efficiency.

6.7 Search and Filter Implementation

Search and filter functionality are major features of the timetable viewer. They make it possible for students and faculty members to locate specific timetable entries quickly. According to the report, users can filter schedules by entering keywords, and the dataset is filtered dynamically based on matching subjects, teachers, rooms, or day constraints. The report also lists subject-wise filtering, faculty filtering, classroom filtering, and advanced search with autocomplete suggestions and history recall logs.

The search and filter process generally works in the following way:

1. The user enters a keyword or selects a filter.
2. The application captures the input using React state.
3. The timetable array is processed using filter logic.
4. Matching records are displayed.
5. Non-matching records are hidden.

This implementation is effective because it helps users avoid manually scanning the entire timetable. For example, if a student wants to find all DBMS lectures, the system displays only relevant slots. If a faculty member wants to locate a classroom or teaching schedule, the filter returns only the matching information.

The search feature improves usability, while the subject/faculty/classroom filters improve precision. Together, these functions make the system much more practical than a static timetable chart.

6.8 Theme Management System

Theme management is one of the most visually important parts of the application. The timetable viewer supports dark/light theme switching along with custom color themes such as Violet, Emerald, Cyberpunk, and Amber. The report explicitly states that theme variables are changed by setting HTML attributes like `data-theme="light"` and `data-color-theme="emerald"`, which instantly swap root-level CSS variable tokens.

Theme management is implemented by updating CSS variables dynamically. This means the layout and styling remain the same, but the color palette changes based on user selection. This approach has several benefits:

- It improves visual comfort.

- It supports different user preferences.
- It reduces eye strain.
- It gives the interface a premium appearance.
- It makes the system feel modern and interactive.

The use of theme switching is especially valuable in long-use academic applications because students may use the timetable system for extended periods. Dark mode can be easier on the eyes in low-light environments, while light mode is useful in bright conditions.

The report also mentions premium frosted glassmorphism effects and micro-animations, which contribute to the overall design quality of the theme system.

.

6.9 Responsive Design Implementation

Responsive design ensures that the timetable viewer works properly on desktops, laptops, tablets, and mobile phones. This is a critical requirement because users may access the system from different devices. The project report explicitly mentions mobile-friendly design using CSS Flexbox and media queries. It also highlights responsive design as one of the implemented features.

.

The responsive implementation likely includes:

- Flexible grid layouts for weekly timetable view
- Vertical stacking for smaller screens
- Adjustable font sizes
- Adaptive spacing and padding
- Mobile-friendly buttons and cards
- Scrollable timetable sections where needed

The project uses CSS3 techniques to ensure that the interface remains readable and usable at different screen sizes. This is especially important for the daily timetable view and filter panels, which must stay accessible even on narrow screens.

Responsive design improves usability because users do not need to zoom or scroll excessively to view the timetable. It also supports the real-world usage pattern of students, who often check schedules on mobile devices while moving between classes.

The report's mention of a modern user interface and responsive design confirms that the project was built with cross-device accessibility in mind.

6.10 Attendance Tracker Implementation

The attendance tracker is an additional academic feature that makes the timetable viewer more useful than a basic schedule display tool. According to the report, the attendance tracker includes subject increments, status progress indicators, and a goal calculator that predicts the number of classes needed to reach a 75% attendance target.

.

The implementation of the attendance tracker likely involves the following steps:

1. The user selects a subject.
2. Attendance counts are updated using buttons or controls.
3. The system calculates the percentage of classes attended.
4. The goal calculator estimates how many more classes are needed to reach the target.
5. Progress indicators display the current status visually.

This feature is beneficial because attendance is a major concern for students in most colleges. By including an attendance module, the timetable viewer becomes more than just an information dashboard; it becomes a planning and self-monitoring tool.

The attendance tracker also encourages students to maintain better class attendance by showing them their current progress in a simple and visual format. Since the logic section of the report mentions that goal calculations are performed when users increment or decrement attendance registers, this feature is clearly integrated into the application workflow.

CHAPTER 7: ALGORITHMS AND LOGIC

Algorithms form the backbone of any software application because they define how data is processed, manipulated, and displayed to users. In the **College Timetable Viewer using React, HTML, and CSS**, algorithms are responsible for loading timetable information, displaying daily and weekly schedules, filtering data, searching records, calculating attendance goals, and switching themes dynamically. The project uses React state management and structured timetable objects to perform these operations efficiently. According to the project report, timetable information is stored in centralized objects, filtered dynamically, and rendered using React components and state-based updates. The application also supports attendance calculations and theme switching through JavaScript logic.

.

The algorithms implemented in this project are designed to provide fast performance, real-time updates, and an intuitive user experience. Each algorithm processes specific user requests and ensures that the timetable viewer remains responsive and easy to use.

7.1 Timetable Loading Algorithm

The timetable loading algorithm is responsible for retrieving timetable information from the centralized data structure and displaying it on the user interface. Since timetable data is stored

inside structured JavaScript objects, the algorithm reads this information and passes it to React components for rendering.

When the application starts, the timetable dataset is loaded into memory. The system then identifies the selected view (daily or weekly) and retrieves the required records. React maps the timetable data and creates user interface elements dynamically. According to the project report, timetable information such as subjects, faculty names, classrooms, timings, and colors is stored inside timetableData.js, and React components dynamically render the timetable using mapping operations.

Steps of Timetable Loading Algorithm

1. Start application.
2. Load timetable dataset.
3. Retrieve timetable object.
4. Check selected timetable view.
5. Pass timetable data to React component.
6. Generate timetable cards or grid cells.
7. Display timetable on screen.
8. End process.

Pseudocode

BEGIN

Load TIMETABLE data

IF data available THEN

 Render timetable component

 Display timetable entries

ELSE

 Show "No Data Available"

ENDIF

END

Advantages

- Fast timetable rendering.
- Easy data management.
- Dynamic updates without page reloads.
- Supports future expansion.

The timetable loading algorithm serves as the foundation for all other timetable operations.

7.2 Daily View Algorithm

The daily view algorithm displays timetable information for a single selected day. This feature allows students and faculty members to focus only on the schedule relevant to the chosen day rather than viewing the entire week.

The report states that the system includes a chronological vertical timeline for day-specific schedules and uses selectors for choosing individual days. The algorithm retrieves timetable entries corresponding to the selected day and displays them in order of time slots.

.

Working Principle

When a user selects a day such as Monday or Tuesday:

- The selected day is stored in React state.
- The timetable dataset is searched for matching entries.
- Only records belonging to that day are extracted.
- The records are sorted according to time.
- The interface updates automatically.

Steps of Daily View Algorithm

1. Receive selected day.
2. Search timetable dataset.
3. Match entries for selected day.
4. Sort entries by time.
5. Generate daily schedule cards.
6. Display timetable.
7. End process.

Pseudocode

BEGIN

Read selectedDay

FOR each timetable entry

 IF entry.day = selectedDay

 Add entry to dailySchedule

 ENDIF

ENDFOR

Display dailySchedule

END

Benefits

- Reduces information overload.
 - Improves readability.
 - Helps students focus on one day at a time.
 - Faster schedule lookup.
-

7.3 Weekly View Algorithm

The weekly view algorithm displays the timetable for the entire academic week. It organizes subjects and time slots into a structured grid layout where each day occupies a separate column.

The project report identifies the weekly timetable view as one of the primary features of the application. The weekly view uses a responsive grid that maps timetable slots from Monday through Saturday.

.

Working Principle

The algorithm retrieves timetable data for all days and arranges it into rows and columns. Each row represents a lecture slot, while each column represents a day.

Steps of Weekly View Algorithm

1. Load complete timetable dataset.
2. Retrieve all timetable entries.
3. Group entries by day.
4. Organize entries into grid structure.
5. Generate weekly timetable table.
6. Display timetable grid.
7. End process.

Pseudocode

BEGIN

FOR each day in week

 Retrieve day schedule

 FOR each slot

 Display timetable cell

 ENDFOR

ENDFOR

END

Benefits

- Provides complete weekly overview.
- Supports academic planning.
- Helps identify schedule conflicts.
- Easy visualization of weekly workload.

The weekly view is especially useful for faculty members and students planning their entire week.

7.4 Search Algorithm

The search algorithm allows users to locate timetable information quickly by entering keywords. The project report mentions advanced search functionality with autocomplete suggestions and history recall logs.

.

The search algorithm works by comparing the user's query against timetable records such as:

- Subject name
- Faculty name
- Classroom name
- Day information

Working Principle

Whenever a user enters text into the search field:

- Input is captured using React state.
- The dataset is scanned.
- Matching records are identified.
- Results are displayed instantly.

Steps of Search Algorithm

1. Receive search query.
2. Convert query to lowercase.
3. Scan timetable records.
4. Compare query with record fields.

5. Collect matching results.
6. Display results.
7. End process.

Pseudocode

BEGIN

Read query

FOR each timetable record

 IF query matches subject

 Display record

 ELSE IF query matches faculty

 Display record

 ELSE IF query matches room

 Display record

 ENDIF

ENDFOR

END

Advantages

- Fast information retrieval.
- Reduces manual searching.
- Improves user experience.
- Supports large timetable datasets.

7.5 Subject Filter Algorithm

The subject filter algorithm enables users to view timetable entries belonging to a specific subject. The project report identifies subject-wise filtering as one of the implemented features.

.

Working Principle

When a user selects or enters a subject name:

- The algorithm scans all timetable entries.
- It checks whether the subject matches.
- Matching entries are displayed.
- Non-matching entries are hidden.

Steps of Subject Filter Algorithm

1. Read subject name.
2. Search timetable dataset.
3. Compare subject fields.
4. Store matching records.
5. Display filtered timetable.
6. End process.

Pseudocode

BEGIN

Read subjectName

FOR each record

 IF record.subject = subjectName
 Display record
 ENDIF

ENDFOR

END

Benefits

- Quick subject identification.
- Helps students track specific courses.
- Simplifies timetable navigation.
- Improves accessibility.

7.6 Faculty Filter Algorithm

The faculty filter algorithm allows users to locate schedules associated with a particular faculty member. The report includes faculty filtering and a faculty directory containing role, contact, office hours, and availability details.

.

Working Principle

The algorithm compares the selected faculty name against timetable entries.

If a match is found:

- The related timetable slots are retrieved.

- Faculty information is displayed.
- Schedule details become visible.

Steps of Faculty Filter Algorithm

1. Receive faculty name.
2. Search timetable records.
3. Match faculty field.
4. Retrieve relevant timetable slots.
5. Display filtered results.
6. End process.

Pseudocode

BEGIN

Read facultyName

FOR each timetable record

 IF record.faculty = facultyName

 Display record

 ENDIF

ENDFOR

END

Advantages

- Helps faculty locate schedules.
- Useful for students seeking teacher information.
- Improves timetable organization.
- Reduces search time.

7.7 Attendance Goal Calculator Algorithm

The attendance goal calculator is one of the advanced features of the timetable viewer. The report mentions attendance tracking, progress indicators, and a goal calculator that predicts the number of classes needed to achieve a target attendance percentage of 75%.

.

Working Principle

The algorithm calculates attendance percentage using:

$$Attendance = \frac{Attended\ Classes}{Total\ Classes} \times 100$$

$$Attendance = \frac{Attended\ Classes}{Total\ Classes} \times 100$$

If attendance is below the target:

- The algorithm estimates additional classes required.
- Progress indicators are updated.

Steps of Attendance Goal Calculator

1. Read attended classes.
2. Read total classes.
3. Calculate attendance percentage.
4. Compare with target percentage.
5. Determine required future classes.
6. Display result.
7. End process.

Pseudocode

BEGIN

attendance =
(attendedClasses / totalClasses) * 100

IF attendance < 75
 Calculate classesNeeded
ENDIF

Display attendance
Display classesNeeded

END

Benefits

- Encourages attendance improvement.
- Helps students achieve eligibility criteria.
- Provides instant feedback.
- Simplifies attendance monitoring.

7.8 Theme Switching Algorithm

The theme switching algorithm manages the visual appearance of the application. According to the report, the project supports Dark Mode, Light Mode, Violet Theme, Emerald Theme, Cyberpunk Theme, and Amber Theme. Theme changes occur by updating HTML attributes and CSS variables dynamically.

.

Working Principle

When a user selects a theme:

- The theme value is stored.
- HTML attributes are updated.
- CSS variables change automatically.
- Interface colors refresh instantly.

Steps of Theme Switching Algorithm

1. Receive selected theme.
2. Update theme state.
3. Set HTML theme attribute.
4. Apply CSS variable values.
5. Refresh interface appearance.
6. Save preference.
7. End process.

Pseudocode

BEGIN

Read selectedTheme

Set themeState = selectedTheme

Apply CSS variables

Save theme preference

Update UI

END

Benefits

- Personalized user experience.
- Improved readability.

- Reduced eye strain.
- Modern application appearance.

The project report specifically states that theme management uses state variables to toggle HTML attributes such as `data-theme="light"` and `data-color-theme="emerald"`, resulting in instant visual updates.

CHAPTER 8: USER INTERFACE DESIGN

User Interface (UI) design is one of the most important parts of any web application because it determines how users interact with the system and how easily they can access its features. In the **College Timetable Viewer using React, HTML, and CSS**, the interface has been designed to be modern, responsive, visually clear, and easy to use for both students and faculty members. The project report highlights several interface features such as weekly and daily timetable views, subject filtering, faculty and classroom directories, dark/light theme support, responsive design, dashboard widgets, and premium glassmorphism styling. These UI elements were included to make the timetable viewer more practical and attractive than a traditional static timetable display.

.

The user interface of the system is not only about appearance; it also supports usability, accessibility, and fast interaction. Every screen in the application is designed to help users find information quickly without confusion. The dashboard provides an overview of the timetable system, while the weekly and daily views organize schedule data in different formats. Filter interfaces make search easier, directory sections provide academic details, dark theme options improve comfort, and the responsive layout ensures that the system works smoothly on mobile devices as well. This chapter explains the design of each major UI section in detail.

8.1 Dashboard Design

The dashboard is the first screen that users see when they open the timetable viewer. Its purpose is to provide a quick overview of the system and present the most important timetable-related information in a simple and organized format. According to the project report, the dashboard includes summary widgets, notifications noticeboard, and sticky notes, which makes it more useful than a plain timetable page.

.

The dashboard is designed to act as the central control panel of the application. From here, users can move to weekly timetable view, daily timetable view, filters, faculty information, classroom details, and attendance tracking. A good dashboard should always reduce the time needed to reach important sections, and this project follows that principle.

The layout of the dashboard is clean and structured. It likely includes a top header area for the title and theme controls, followed by summary cards or widgets that highlight useful academic information. These may include current day, current subject, attendance status, or upcoming lecture details. The use of cards and widgets makes the interface visually balanced and easier to scan.

Another important aspect of dashboard design is clarity. Students should be able to quickly understand what the system offers without reading long instructions. The dashboard therefore uses short labels, visual grouping, and attractive spacing to guide the user naturally. Since the report mentions premium glassmorphism effects and micro-animations, the dashboard likely also includes soft shadows, transparent panels, and subtle motion effects to make it appear modern and polished.

.

The dashboard also supports usability by serving as a launch point for the rest of the system. This means the design is not only decorative but also functional. A well-designed dashboard improves user confidence and creates a strong first impression of the application.

8.2 Weekly Timetable Screen

The weekly timetable screen is one of the most important parts of the user interface because it gives users a complete overview of the entire academic week. The project report identifies the weekly timetable view as a responsive grid mapping Monday through Saturday slots. This means the screen is organized in a tabular or grid format where each day and time slot is clearly visible.

.

The weekly screen is especially useful for students who want to understand their full class schedule at a glance. Instead of checking one lecture at a time, they can see the complete pattern of classes across the week. This helps with planning, attendance management, assignment preparation, and travel scheduling.

The interface is designed to be compact yet readable. Each timetable cell likely contains subject name, faculty name, room number, and lecture type. To improve readability, different subject categories are color-coded, such as lectures, labs, and seminars. This makes the weekly view easier to scan quickly, especially when many classes are displayed on the same screen.

.

A major challenge in weekly timetable design is fitting a lot of information into a limited space without making it look crowded. The project addresses this by using clean spacing, grid alignment, and responsive design. The layout adapts to the screen size so that users can still read the timetable comfortably on smaller devices.

The weekly timetable screen also benefits from visual hierarchy. Important items such as subject names or time slots are likely displayed more prominently than secondary information like room numbers or faculty names. This helps users focus on the most useful details first.

Since the application is built with React, the weekly timetable screen can update dynamically when filters are applied or when the selected day changes. This makes the view more interactive than a static timetable image or PDF sheet. The weekly screen therefore represents both the structure and the intelligence of the system.

8.3 Daily Timetable Screen

The daily timetable screen provides a focused view of the schedule for one selected day. The project report describes this as a chronological vertical timeline card layout with day-specific selectors. This design is very useful because it reduces information overload and allows users to concentrate on one day's classes at a time.

.

The daily view is ideal for students who only need to check today's schedule or faculty members who want to review a specific day quickly. Instead of presenting the full week, the interface displays only the relevant time slots for the selected day. This makes the page cleaner and easier to understand.

The vertical timeline style is a strong design choice because it follows the natural way users think about time. Lecture cards are usually arranged from top to bottom in time order, which helps users see the sequence of classes clearly. Each card may display the subject, faculty, room, and any special notes or status indicators.

The daily screen also supports navigation controls such as next day and previous day buttons. These controls allow users to move between days without returning to the dashboard or weekly timetable. This improves speed and convenience. The report explicitly mentions day navigation and quick day selectors, confirming that this part of the interface was designed for easy movement.

.

Another benefit of the daily view is that it allows more spacing per lecture entry. Unlike the weekly table, which must fit many days into one layout, the daily page can show more detail for each class. This makes it possible to highlight attendance notes, room changes, or special announcements if needed.

The daily timetable interface is therefore both practical and user-friendly. It gives users a precise look at one day's schedule while maintaining the same visual identity as the rest of the application.

8.4 Subject Filter Interface

The subject filter interface is designed to help users quickly find timetable entries related to a specific subject. The project report includes subject-wise filtering and advanced search functionality with autocomplete suggestions and history recall logs. This indicates that the interface is not just a simple text box but a smart and interactive search tool.

.

The layout of the subject filter interface should be simple and prominent so that users can find it easily. Usually, search fields are placed near the top of the page or inside a dedicated filter panel. The user can type a subject name, and the system immediately displays matching timetable entries.

A well-designed filter interface reduces effort and improves accuracy. Instead of scanning the whole timetable manually, users can search for "DBMS," "Mathematics," or any other subject

and instantly see all related slots. This is especially useful in large colleges where multiple subjects appear across different days and sections.

The interface likely supports both typing and selection-based interaction. Autocomplete suggestions make it easier to enter subject names correctly, while history recall logs help users repeat previous searches quickly. This is especially helpful if the same subject is searched often.

Visually, the filter interface should stand out from the rest of the page but still match the application theme. Rounded input boxes, clear labels, and subtle icons can improve usability. The design must remain uncluttered so that the filter feels like a quick tool rather than a complex form.

The subject filter interface is a key example of how the application combines design and functionality. It transforms timetable access from a static viewing experience into a dynamic search-based experience.

8.5 Faculty Directory Interface

The faculty directory interface provides a structured way to view faculty-related information. The project report states that the faculty filtering section includes role, contact, office hours, and availability details. This means the interface acts like a compact academic directory within the timetable system.

.

The faculty interface should present information clearly and professionally. Since faculty details are academic and important, the layout should use neat cards or list rows with sufficient spacing and readable typography. Each faculty card may show the teacher's name, department, designation, subject expertise, contact information, and office timing.

A directory-style UI is useful because it gives timetable context. Students do not only see class timings; they also know who teaches the class and when that teacher is available for consultation. This improves communication and helps users plan meetings more effectively.

From a design point of view, the faculty section should avoid overcrowding. A clean card layout with icons, labels, and aligned text works better than a plain text table. If color coding is used, it should remain subtle and professional.

The report shows that the system includes faculty filtering as one of the main features, which means the interface must also support search or selection by faculty name.

.

This makes the directory more interactive and useful for both students and faculty members.

The faculty directory interface adds depth to the timetable viewer because it turns the application into a mini academic information portal rather than a basic schedule chart.

8.6 Classroom Directory Interface

The classroom directory interface is designed to show room and laboratory information in a clear, accessible form. The report states that the project includes classroom and laboratory lists detailing capacity, building, and equipment. This makes the interface valuable for students and faculty who need to know where a lecture or practical session will be held.

.

A classroom interface should be highly organized because room details often include multiple fields such as room number, building block, seating capacity, lab type, and available equipment. Presenting these details in a card or table format helps users quickly understand room information without confusion.

The design should support quick comparison between classrooms. For example, users may want to know which room has more capacity or which lab contains a specific setup. Clean labels and aligned data improve readability.

Since timetable changes can sometimes involve room substitutions, the classroom directory should be easy to update and visually consistent with the rest of the application. The interface should also support classroom filtering so that users can find classes associated with a particular room. This improves the overall utility of the timetable system.

The classroom section is especially useful in technical colleges or universities where laboratory sessions and equipment-based classes are common. By including classroom and lab details, the application becomes more complete and practical.

8.7 Dark Theme Design

Dark theme design is an important part of the user interface because it provides comfort, modern aesthetics, and better visual accessibility in low-light conditions. The project report explicitly mentions dark/light theme support and custom themes such as Violet, Emerald, Cyberpunk, and Amber.

.

The dark theme in this application likely uses deeper background shades with contrasting text and highlighted panels. This creates a premium appearance and reduces eye strain for users who study or work on the timetable at night. Dark mode is also popular among modern users because it feels sleek and professional.

A good dark theme must maintain readability. Text should remain bright enough to be clearly visible, while buttons, cards, and containers should preserve contrast. The use of soft shadows and subtle borders helps separate interface sections without making the layout harsh.

The project's glassmorphism styling works particularly well with dark theme design. Transparent panels with blur effects and glowing accents create a futuristic appearance that matches the cyber-inspired custom themes mentioned in the report.

.

Theme switching also improves personalization. Some users prefer light mode during the day and dark mode at night, while others may choose a color theme based on personal taste. The interface supports this flexibility by allowing quick theme changes without reloading the page.

The dark theme design therefore contributes both to aesthetics and usability. It makes the application feel modern while also improving long-duration reading comfort.

8.8 Mobile Responsive Layout

Mobile responsive layout is one of the most important parts of the UI design because many students access academic information from their phones. The project report mentions that the application is mobile-friendly and uses CSS Flexbox and media queries to adapt to different screen sizes.

.

A responsive layout ensures that the timetable viewer remains usable on small screens without losing clarity. On a mobile phone, the weekly grid may need to stack vertically or become horizontally scrollable, while the daily view can be shown as a clean card list. Buttons and filters should also resize properly so users can tap them easily with their fingers.

The responsive design of the application likely includes flexible width containers, proportional spacing, and adaptive typography. These elements help keep the layout balanced on both large and small screens. The interface should avoid tiny text, overlapping cards, or crowded controls because such issues reduce usability on mobile devices.

The advantage of a responsive timetable viewer is that users can check class timings anywhere, whether on the bus, in the hostel, or while walking between lectures. This aligns well with the daily use pattern of students and faculty members.

The report's emphasis on responsive design shows that this feature was considered part of the main system quality, not just an extra improvement.

.

8.9 Glassmorphism Design Concept

Glassmorphism is a modern UI design style that uses transparency, blur, layering, and soft shadows to create a frosted glass effect. The project report specifically mentions premium frosted glassmorphism elements and micro-animations, which means this concept is one of the key visual styles used in the application.

.

The purpose of glassmorphism in this project is to make the timetable viewer feel elegant, premium, and modern. Transparent cards placed over layered backgrounds create depth and visual richness. This style works especially well in dashboard sections, timetable cards, and theme-based interfaces.

The glass effect can improve focus by separating content panels from the background while still maintaining a light, airy appearance. Combined with smooth transitions and subtle motion effects, it gives the application a polished and professional look.

However, glassmorphism must be used carefully. If the transparency is too strong or the blur is too high, the text may become difficult to read. Therefore, the design should balance beauty

and clarity. The report's mention of modern UI and premium styling suggests that this balance was considered during development.

.

Glassmorphism also helps the timetable viewer stand out from ordinary college websites. Instead of using a plain and boring layout, the interface feels like a high-quality web application. This improves user engagement and creates a better impression of the project as a whole.

CHAPTER 9 CODE SECTION FORMAT

The source code of the College Timetable Viewer using React, HTML, and CSS is designed to create a clean, interactive, and responsive timetable management application. The project is built around a modular structure where the main application file, data file, stylesheet, and feature-specific logic work together to provide timetable display, filtering, attendance tracking, theme switching, and print/export support. The report indicates that the project uses App.jsx for the main React interface, timetableData.js for centralized timetable objects, and styles.css for the premium glassmorphism-based UI design.

This chapter explains the purpose and working of the major source code sections. It focuses on how the main application is organized, how timetable data is stored, how UI components are structured, how the stylesheet controls the appearance of the system, and how the search, attendance, theme, and export modules function. The project demonstrates React components, state management, filtering, responsive design, and other modern front-end concepts.

9.1 index.html

Purpose: Entry HTML file used to mount the React application.

HTML

```
<!doctype html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport"
content="width=device-width, initial-scale=1.0" />
<title>College Timetable Viewer</title>
</head>

<body>
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

Explanation:

- Root container for React application.

- Loads React application through main.jsx.
- Provides responsive viewport support.
- Acts as starting point of application.

ALL_PROJECT_CODE

9.2 main.jsx

JavaScript

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './styles.css'
```

```
ReactDOM.createRoot(
  document.getElementById('root')
).render(
  <React.StrictMode>
  <App />
</React.StrictMode>
)
```

Explanation

The file initializes React and mounts the App component to the DOM.

Functions:

- Imports React.
- Loads CSS styles.
- Creates root element.
- Renders App component.
- Enables React Strict Mode.

ALL_PROJECT_CODE

9.3 App.jsx (Main Application)

JavaScript

```
const [activeTab, setActiveTab] =
  useState('dashboard')
```

```
const [theme, setTheme] =
  useState('dark')
```

```
const [favorites, setFavorites] =  
useState(['React JS', 'DBMS'])
```

Explanation

This is the central controller of the application.

Responsibilities:

- Navigation management
- Theme management
- Attendance storage
- Notifications
- Rendering dashboard
- Rendering timetable
- Managing state

Features implemented:

- Dashboard
- Timetable Viewer
- Attendance Tracker
- Faculty Directory
- Academic Hub
- Analytics
- AI Assistant
- Export Module
- Settings Module

ALL_PROJECT_CODE

9.4 Attendance Tracker Code

JavaScript

```
const [attendanceData,  
setAttendanceData] = useState({  
  
'DBMS':  
{  
  attended: 12,  
  total: 15  
},  
},
```

```
'React JS':  
{  
  attended: 14,  
  total: 16  
}  
  
})
```

Formula Used

JavaScript

Attendance Percentage =
(Attended / Total) * 100

Example:

Attended = 12
Total = 15

Attendance =
(12 / 15) × 100

= 80%

Purpose

- Track attendance.
- Calculate percentages.
- Display progress bars.
- Predict attendance goals.

ALL_PROJECT_CODE

9.5 Theme Management Code

JavaScript

```
const [theme,  
setTheme] = useState('dark')
```

```
const [colorTheme,  
setColorTheme] = useState('default')
```

Features

- Dark Mode
- Light Mode

- Violet Theme
- Emerald Theme
- Cyberpunk Theme
- Amber Theme

Theme synchronization:

JavaScript

```
useEffect(() => {
  document.documentElement
    .setAttribute('data-theme',
  theme)
}, [theme])
```

ALL_PROJECT_CODE

9.6 AI Assistant Module

JavaScript

```
const [messages,
  setMessages] = useState([
  {
  sender:'bot',
  text:'Hello! I am your AI Timetable Assistant'
  }
  ])
```

Functions

- Responds to timetable questions.
- Provides faculty information.
- Provides room information.
- Suggests study schedules.
- Detects timetable clashes.

Example Query:

What is my next class?

Output:

DBMS Lecture

09:00 AM

Room A-101

ALL_PROJECT_CODE

9.7 CSS Styling System

CSS

```
:root {  
  
  --bg-primary: #0a0f1d;  
  
  --accent: #8b5cf6;  
  
  --success: #10b981;  
  
  --warning: #f59e0b;  
  
  --danger: #ef4444;  
  
}
```

Purpose

Defines:

- Colors
- Typography
- Shadows
- Layout variables
- Theme variables

Advantages:

- Easy theme switching
- Consistent UI
- Better maintainability

ALL_PROJECT_CODE

9.8 PWA Support

JavaScript

```
if ('serviceWorker' in navigator) {  
  
  navigator.serviceWorker  
    .register('/sw.js')  
  
}
```

Benefits

- Offline support
- Faster loading
- Installation as app
- Better user experience

CHAPTER 10: TESTING AND VALIDATION

Testing and validation are essential stages in software development because they ensure that the developed system works correctly, meets user requirements, and performs reliably under different conditions. For the **College Timetable Viewer using React, HTML, and CSS**, testing was focused on verifying timetable display, search and filter behavior, daily and weekly views, navigation, theme switching, attendance tracking, responsive layout, browser support, and print/export output. The project report confirms that the system includes weekly and daily timetable views, subject-wise filtering, faculty and classroom filters, day navigation, advanced search, responsive design, theme support, attendance tracking, PWA support, and PDF/print export. These implemented features form the basis of the testing strategy.

.

The purpose of testing is not only to identify defects but also to confirm that the application provides a smooth and user-friendly experience. Since the timetable viewer is intended for daily academic use by students and faculty, it must be accurate, fast, readable, and compatible with common browsers and mobile devices. The final result section of the report indicates that the application successfully demonstrates React components, state management, filtering, and responsive web design, which suggests that the implementation was validated through functional and interface-level verification.

.

10.1 Testing Methodology

The testing methodology used for this project follows a combination of manual testing and feature-based validation. Since the application is a front-end web system, most testing was done by interacting with the interface directly and checking whether each function behaved as expected. This approach is suitable for a React-based timetable viewer because the application relies heavily on component rendering, state updates, filtering logic, and CSS responsiveness.

The methodology was divided into several stages. First, the project was checked at the component level to ensure that each section, such as the dashboard, weekly timetable, daily timetable, search module, faculty directory, and attendance tracker, rendered correctly. Second, the interactions between components were tested to make sure state updates triggered the correct UI changes. Third, the application was tested across browsers and screen sizes to verify compatibility and responsiveness. Finally, print/export behavior was reviewed to ensure that timetable output remained clean and readable when generated in print mode. The report mentions media print overrides, service worker caching, and responsive design, which

confirms that these areas were part of the overall system design and therefore needed validation.

Testing was carried out using actual user actions such as entering search terms, changing timetable views, selecting different days, switching themes, and updating attendance values. The expected result was compared with the actual result in each case. This helped confirm that the interface was not only visually correct but also logically accurate.

A good testing methodology for this type of project must verify both functionality and usability. A timetable viewer may display data correctly but still fail if filters are slow, layouts are confusing, or the mobile screen becomes unreadable. Therefore, testing was performed with the goal of checking correctness, performance, and user comfort together.

10.2 Unit Testing

Unit testing focuses on individual modules or small pieces of functionality. In this project, unit testing was applied to the core logic functions that control timetable loading, search matching, filtering, attendance calculation, and theme switching. Since the timetable data is stored in structured JavaScript objects inside `timetableData.js`, it is suitable for small logical tests that check whether a given input produces the correct output. The report explains that timetable information is stored in centralized structured objects and that filters are applied dynamically based on matching subjects, teachers, rooms, or day constraints.

.

A few examples of unit-level checks are:

The timetable loader should correctly retrieve timetable records for a given day and slot.

The subject filter should return only those entries where the subject matches the search term.

The faculty filter should display only timetable entries associated with the selected faculty member.

The attendance calculator should compute percentage values correctly and predict how many classes are needed to reach the target.

The theme manager should update the selected theme state and apply the matching CSS variables.

The main purpose of unit testing in this project is to confirm that each logical function behaves predictably before it is connected to the rest of the interface. Since the application uses React state updates, even a small logic error can affect the displayed timetable. For example, if a filter function does not correctly compare subject names, the user may see incomplete or incorrect results.

Unit testing is especially useful for future modifications. If additional features are added later, such as calendar integration or notification support, the developer can test the new logic separately without disturbing the rest of the application. This increases confidence in the stability of the system.

10.3 Integration Testing

Integration testing verifies whether different parts of the system work together correctly. In the timetable viewer, this is important because the interface is composed of several connected modules: timetable display, filters, navigation, attendance tracking, theme switching, dashboard cards, and export support. The project report shows that the application uses React components, state routers, local storage for settings, and dynamic rendering, which means integration behavior is central to the system.

For example, when a user enters a search query, the search component must communicate with the timetable data and the rendered result area. When a user changes the selected day, the navigation controls must update the state and the timetable view must refresh automatically. When a user changes the theme, the theme state must connect to CSS variables so that the interface appearance changes instantly. The report mentions that state variables toggle HTML attributes and swap root-level CSS variable tokens instantly, which is exactly the kind of interaction that integration testing must verify.

.

Integration testing in this project confirmed that:

Search input updates the timetable display correctly.

Subject, faculty, and classroom filters can work together without conflicts.

Daily and weekly views can be switched without breaking the layout.

Attendance counters update progress indicators properly.

Theme changes apply consistently across the dashboard and timetable pages.

Print/export mode produces a clean timetable view without unnecessary interface elements.

The success of integration testing shows that the application is not just a collection of separate components, but a coordinated system in which the data layer, logic layer, and user interface layer function together smoothly.

10.4 User Interface Testing

User interface testing ensures that the visual design and interaction flow are correct, readable, and easy to use. Since the timetable viewer is a front-end application, UI testing is one of the most important parts of validation. The report mentions a modern user interface with premium frosted glassmorphism elements, micro-animations, dashboard widgets, dark/light theme support, and responsive design. These elements are visual features that must be checked carefully during testing.

UI testing focused on the following points:

Whether all text is readable and aligned properly.

Whether timetable cards and grid cells display data in a clean format.

Whether buttons, dropdowns, and filter fields are placed in intuitive locations.

Whether color coding helps users distinguish lectures, labs, and seminars.

Whether dark mode provides enough contrast for comfortable reading.

Whether dashboard widgets and notices remain visually balanced.

Whether print-friendly output still looks organized when viewed in the browser print preview.

The daily timetable and weekly timetable screens were especially important in UI testing because they display dense academic information. The weekly grid had to remain clear even when many classes were shown together, while the daily view had to preserve spacing and readability in card format. The project report confirms that the application includes both of these views, which supports the need for structured UI validation.

UI testing also verified the visual behavior of the subject filter and faculty/classroom filters. These sections need to feel responsive and simple, since users will use them often. The report includes output screens for subject filter functionality and faculty/classroom filters, which indicates that these interfaces were implemented as visible parts of the application and therefore had to be tested in the final build.

10.5 Browser Compatibility Testing

Browser compatibility testing checks whether the application works correctly in different web browsers. This is important because users may access the timetable viewer using Chrome, Edge, Firefox, or other modern browsers. The project report lists Chrome, Edge, and Firefox under software requirements, showing that compatibility across common browsers was expected from the beginning.

.

In browser compatibility testing, the following aspects were checked:

Whether the application loads correctly in the browser.

Whether React components render without layout issues.

Whether CSS styles, shadows, and themes appear consistently.

Whether search and filter behavior is the same across browsers.

Whether the print preview and export mode remain readable.

Whether local storage-based settings such as themes and saved preferences continue to work.

The timetable viewer uses standard web technologies such as React, HTML5, CSS3, and JavaScript, which are well supported by modern browsers. This reduces the risk of compatibility problems. However, testing is still necessary because different browsers may handle spacing, fonts, transitions, and print settings slightly differently.

The project's final result states that the application provides an efficient way to view, search, and filter class schedules and demonstrates React components, state management, filtering, and responsive web design. That result suggests the app behaved consistently enough across the tested environment to be considered successful.

.

Browser compatibility testing ensures that users get the same core experience regardless of the browser they prefer. This is especially important in academic settings where students may use different devices and operating systems.

10.6 Mobile Responsiveness Testing

Mobile responsiveness testing verifies that the timetable viewer remains usable on smaller screens such as smartphones and tablets. Since the project report highlights mobile-friendly layout using CSS Flexbox and media queries, responsiveness is one of the project’s major design goals.

.

The application was tested to ensure that:

The dashboard does not overflow on smaller screens.

The weekly timetable can be viewed without losing readability.

The daily timetable cards stack neatly and remain easy to scroll.

The search bar and filter controls remain accessible on mobile.

Buttons are large enough to tap comfortably.

Text remains legible without excessive zooming.

Theme switching works properly on small screens.

Responsive testing is particularly important for the weekly timetable screen because it contains many columns and slots. On mobile devices, this view may need horizontal scrolling, condensed cells, or an alternate layout to preserve clarity. The daily timetable screen is usually more mobile-friendly because it uses vertical cards, and the report’s description of daily view as a chronological vertical timeline confirms that this layout is well suited for smaller devices.

Testing on mobile screens also checked whether the glassmorphism visual style remained usable and whether the dark theme maintained contrast on compact displays. The result was that the interface stayed readable and functional, supporting the project’s claim of a clean, responsive, and interactive experience.

10.7 Test Cases and Results

The following table summarizes representative test cases used during validation of the timetable viewer.

Test Case ID	Test Scenario	Input / Action	Expected Result	Actual Result	Status
TC-01	Load dashboard	Open application	Dashboard and overview widgets appear	Dashboard loaded correctly	Pass
TC-02	Weekly timetable view	Select weekly view	Complete week timetable grid is displayed	Grid displayed with correct layout	Pass
TC-03	Daily timetable view	Select a day	Only selected day timetable appears	Correct daily schedule shown	Pass
TC-04	Subject search	Enter subject name	Matching entries are filtered	Relevant entries returned	Pass
TC-05	Faculty filter	Choose faculty name	Related timetable entries appear	Correct faculty schedule shown	Pass
TC-06	Classroom filter	Choose room/lab	Room-based timetable entries appear	Correct classroom data shown	Pass
TC-07	Theme change	Switch to dark mode	Interface changes theme instantly	Theme updated correctly	Pass
TC-08	Attendance update	Increase attended classes	Percentage and goal update	Attendance calculated properly	Pass
TC-09	Browser check	Open in Chrome/Firefox	Interface works consistently	Compatible in tested browsers	Pass
TC-10	Mobile view	Open on small screen	Layout remains readable	Responsive layout worked	Pass
TC-11	Print/export preview	Print timetable	Clean landscape timetable output	Output formatted correctly	Pass

The test cases above are representative of the main functions described in the project report, including weekly and daily timetable views, subject filtering, faculty and classroom directories, responsive design, theme support, attendance tracking, and print/export support.

The test results showed that the system performed well in all major scenarios. The timetable data was displayed correctly, filters worked as intended, theme changes were immediate, and the responsive interface adapted properly to different screen sizes. The final report also states

that the project was successfully developed and provides an efficient way to view, search, and filter class schedules, which confirms the overall success of the testing and validation stage.

CHAPTER 11: RESULTS AND DISCUSSION

The **College Timetable Viewer using React, HTML, and CSS** was successfully developed and tested to provide a modern, responsive, and interactive timetable management solution. The application integrates multiple features such as weekly and daily timetable views, subject filtering, faculty and classroom directories, attendance tracking, dashboard summaries, theme customization, and print/export support. The results obtained from the implementation demonstrate that the system effectively meets its objectives of improving timetable accessibility, simplifying schedule management, and enhancing user experience.

The project report confirms that the application provides an efficient way to view, search, and filter class schedules while demonstrating the practical use of React components, state management, filtering mechanisms, and responsive web design. The discussion presented in this chapter evaluates the outputs generated by the system and analyzes the performance and effectiveness of each major feature.

.

11.1 Dashboard Output

The dashboard serves as the main entry point of the application and provides users with a quick overview of timetable-related information. According to the project report, the dashboard includes summary widgets, notifications, noticeboard sections, and sticky notes. This makes it significantly more useful than a traditional timetable display because it combines schedule information with additional academic support features.

.

Result Analysis

When the application starts, the dashboard is displayed successfully and acts as a central control panel for the system. Users can access all major functions such as:

- Weekly timetable view
- Daily timetable view
- Subject filters
- Faculty directory
- Classroom directory
- Attendance tracker
- Theme controls

The dashboard organizes information using cards and widgets, making navigation intuitive and visually appealing. The dashboard output demonstrates that users can quickly access essential timetable functions without navigating through multiple screens.

Discussion

The dashboard significantly improves usability by reducing the number of clicks required to access timetable information. Instead of opening separate pages, users can reach important features directly from the main screen. The inclusion of modern UI elements and dashboard summaries enhances the overall user experience and provides a professional appearance.

The successful implementation of the dashboard confirms that the application achieves one of its primary objectives: providing centralized access to academic schedule information.

.

11.2 Weekly View Output

The weekly timetable view is one of the core features of the application. The project report describes it as a responsive grid that maps timetable slots from Monday through Saturday. This screen provides users with a complete overview of their academic schedule for the entire week.

.

Result Analysis

The weekly timetable was displayed successfully in a structured grid format. Each timetable cell contained relevant information such as:

- Subject name
- Faculty name
- Room number
- Lecture type
- Color category

Different subject categories were color-coded to improve readability and allow users to identify lectures, practicals, and seminars quickly.

Discussion

The weekly timetable view proved highly effective for academic planning. Students can analyze their workload across the entire week, while faculty members can review teaching schedules efficiently. The responsive grid layout remains organized even when multiple timetable entries are displayed simultaneously.

Another important observation is that the weekly view helps users identify scheduling patterns and possible conflicts. Since all timetable data is visible at once, users can manage their time more effectively.

The successful output of the weekly timetable demonstrates that the React rendering logic and timetable data structure function correctly and efficiently.

.

11.3 Daily View Output

The daily timetable view focuses on displaying schedule information for a single selected day. The report explains that this feature uses a chronological vertical timeline card layout with day selectors and navigation controls.

.

Result Analysis

The daily timetable output displayed all lectures for the selected day in chronological order. Each lecture appeared inside a separate card containing:

- Subject details
- Faculty information
- Room number
- Time slot
- Category color

Navigation buttons allowed users to move between days quickly.

Discussion

The daily view reduces information overload by presenting only the relevant schedule for one day. This makes it particularly useful for students who need to check today's classes before attending lectures.

The card-based design also improves readability because each lecture receives adequate spacing. Users can easily focus on one class at a time without scanning a large timetable grid.

The successful implementation of the daily timetable confirms that day selection, navigation logic, and React state updates are functioning correctly. The feature effectively complements the weekly view by providing a more focused schedule presentation.

.

11.4 Subject Filter Results

Subject filtering is one of the most practical features of the timetable viewer. The project report identifies subject-wise filtering as a major implemented feature that allows users to locate specific timetable entries quickly.

.

Result Analysis

The subject filter successfully displayed timetable entries corresponding to the selected subject. When users entered a subject name:

- Matching timetable records appeared instantly.
- Non-matching records were hidden.
- Results updated dynamically without page reloads.

The filter responded quickly and accurately to user input.

Discussion

The results indicate that the filtering algorithm works efficiently with the centralized timetable dataset. Students can locate all lectures associated with a particular subject without manually searching the entire timetable.

The filter improves productivity by reducing search time and making timetable navigation more convenient. It also demonstrates the effectiveness of React's state management and dynamic rendering capabilities.

The successful performance of the subject filter confirms that the search and filtering modules achieve their intended objectives.

.

11.5 Faculty Directory Results

The faculty directory provides detailed information about faculty members, including roles, contact details, office hours, and availability. The report lists faculty filtering and faculty information display as key project features.

.

Result Analysis

The faculty directory successfully displayed:

- Faculty names
- Designations
- Contact information
- Office hours
- Teaching schedules

Faculty filtering also worked correctly, allowing users to retrieve information about specific faculty members.

Discussion

The faculty directory improves communication between students and teachers by making important academic information easily accessible. Students can quickly identify who teaches a particular subject and determine faculty availability when assistance is required.

The results demonstrate that the faculty data structure and filtering mechanisms operate correctly. The directory transforms the timetable viewer from a simple schedule display into a more comprehensive academic information portal.

Overall, the faculty directory contributes significantly to the usefulness and completeness of the application.

.

11.6 Attendance Tracker Results

The attendance tracker is one of the advanced features of the system. According to the project report, it includes attendance counters, progress indicators, and a goal calculator that predicts the number of classes needed to achieve a 75% attendance target.

.

Result Analysis

The attendance tracker successfully performed the following tasks:

- Recorded attendance values
- Calculated attendance percentages
- Displayed progress indicators
- Predicted required classes for attendance goals

Attendance calculations updated instantly whenever users modified attendance records.

Discussion

The attendance tracker proved highly useful because it provides actionable academic information rather than merely displaying timetable data. Students can monitor their attendance status continuously and make informed decisions about class participation.

The goal calculator is particularly valuable because it helps students understand how many additional classes they must attend to maintain eligibility requirements.

The successful implementation of the attendance tracker demonstrates the effectiveness of the project's state management and calculation logic. It also adds significant educational value to the timetable viewer.

.

11.7 Theme Switching Results

Theme customization is an important user experience feature included in the project. The report states that the application supports Light Mode, Dark Mode, Violet Theme, Emerald Theme, Cyberpunk Theme, and Amber Theme.

.

Result Analysis

Theme switching was tested across all available themes.

The results showed that:

- Theme changes occurred instantly.
- CSS variables updated correctly.
- Interface readability remained high.
- Dashboard, timetable views, and filters adapted consistently.

The dark theme provided comfortable viewing in low-light environments, while custom themes enhanced personalization.

Discussion

The theme switching feature significantly improves user satisfaction because it allows users to choose visual styles that match their preferences.

The implementation also demonstrates the successful use of CSS variables and dynamic state management. Since theme changes occur without page reloads, the interface feels modern and responsive.

The results confirm that theme customization is both functional and visually effective. It enhances comfort, aesthetics, and overall user engagement.

.

11.8 Performance Analysis

Performance analysis evaluates how efficiently the timetable viewer operates under normal usage conditions. Since the application relies on React components and structured timetable data, performance depends largely on rendering efficiency, state updates, filtering speed, and responsiveness.

Performance Evaluation Criteria

The following factors were analyzed:

- Timetable loading speed
- Search response time
- Filter performance
- Theme switching speed
- Navigation responsiveness
- Mobile compatibility
- Browser compatibility

Observed Results

The system demonstrated the following characteristics:

- Fast timetable loading.
- Instant state-based updates.
- Smooth navigation between views.
- Responsive filter operations.
- Immediate theme switching.
- Stable browser performance.
- Mobile-friendly interface behavior.

The report confirms that timetable information is rendered dynamically using React state updates and filtering mechanisms, which contributes to efficient performance.

.

Discussion

The use of React significantly improves performance because only affected components are re-rendered when state changes occur. This prevents unnecessary page refreshes and ensures a smooth user experience.

Search and filtering operations were completed quickly because timetable data is stored in structured objects and processed directly within the application. Theme switching also performed efficiently due to the use of CSS variables.

The responsive design further enhances performance by adapting layouts automatically without requiring separate mobile versions of the application.

Overall, the application provides stable and efficient performance across different devices and browsers, demonstrating the suitability of React for timetable management systems.

CHAPTER 12: FUTURE ENHANCEMENTS

The **College Timetable Viewer using React, HTML, and CSS** successfully provides an efficient platform for viewing, managing, and accessing academic schedules. The application includes features such as weekly and daily timetable views, subject filtering, faculty directories, attendance tracking, theme customization, responsive design, and print/export support. While the current system fulfills its primary objectives, there are several opportunities for future improvements that can make the application more intelligent, scalable, and beneficial for educational institutions. The project report itself suggests future enhancements such as AI timetable assistance, Google Calendar synchronization, offline databases, push notifications, and real-time collaboration. These enhancements can transform the timetable viewer from a schedule management tool into a complete academic planning ecosystem.

.

Future enhancements focus on improving automation, communication, accessibility, and decision-making capabilities. The following sections discuss potential upgrades that can significantly increase the usefulness and effectiveness of the system.

12.1 AI-Based Timetable Assistant

Artificial Intelligence (AI) is transforming educational technology by enabling systems to provide intelligent recommendations, automation, and personalized assistance. One of the most promising future enhancements for the College Timetable Viewer is the integration of an AI-Based Timetable Assistant.

The project report identifies AI timetable assistance as a future enhancement involving conversational schedule management and automated timetable updates. Such a feature would allow users to interact with the timetable system using natural language rather than manually searching through schedules.

.

For example, students could ask questions such as:

- "What classes do I have tomorrow?"
- "When is my next DBMS lecture?"
- "Which room is assigned for Operating Systems Lab?"
- "How many classes do I have this week?"

The AI assistant would process the query and provide instant answers.

Faculty members could use the assistant to retrieve their teaching schedules, identify classroom allocations, or receive reminders about upcoming lectures. The assistant could also help timetable coordinators by detecting scheduling conflicts and suggesting alternative arrangements automatically.

An advanced AI system could further analyze attendance patterns, identify missed classes, and recommend study schedules. It could also generate personalized academic summaries based on timetable data and attendance records.

The integration of AI would significantly improve user convenience and make the timetable viewer more interactive and intelligent.

12.2 Push Notifications

Push notifications are another valuable enhancement that can improve communication between the timetable system and its users. The project report specifically mentions the possibility of triggering push alerts before class periods begin.

.

Currently, users must open the timetable application manually to check their schedules. With push notification support, important information could be delivered automatically to users' devices.

Potential notification scenarios include:

- Class starting reminders.

- Classroom change alerts.
- Faculty substitution notifications.
- Timetable modification announcements.
- Attendance warnings.
- Assignment and examination reminders.

For example, the system could send a notification ten minutes before a lecture starts, reminding students about the subject name, classroom location, and faculty member. Similarly, if a room change occurs, students could be informed instantly.

Push notifications would improve punctuality, reduce missed classes, and enhance communication efficiency. Faculty members could also benefit from reminders regarding lectures, meetings, and timetable updates.

Another advantage is that notifications can be personalized. Students would only receive updates related to their specific courses, while faculty members would receive information relevant to their teaching schedules.

This enhancement would transform the timetable viewer into a proactive academic assistant rather than a passive information system.

12.3 Google Calendar Integration

Calendar integration is an increasingly important feature in modern academic management systems. The project report lists Google Calendar synchronization as one of the planned future enhancements.

.

Integrating the timetable viewer with Google Calendar would allow students and faculty members to automatically synchronize their schedules with their personal calendars. This would eliminate the need to manually enter class timings into separate scheduling applications.

Benefits of Google Calendar integration include:

- Automatic timetable synchronization.
- Event reminders.
- Cross-device schedule access.
- Better time management.
- Reduced risk of missing classes.

For example, when a timetable is updated, the corresponding calendar events could be updated automatically. If a lecture is rescheduled or canceled, the calendar would reflect the change immediately.

Calendar integration would also allow users to combine academic schedules with personal appointments, extracurricular activities, and project deadlines. This would create a unified planning environment and improve overall productivity.

In addition, Google Calendar supports sharing and collaboration features. Faculty members could share schedules with colleagues, while students could coordinate group study sessions more effectively.

This enhancement would make timetable management more seamless and integrated into users' daily routines.

12.4 Real-Time Timetable Synchronization

One limitation of static timetable systems is that schedule changes may not be reflected immediately for all users. To address this issue, the project report proposes real-time timetable synchronization through technologies such as WebSockets.

.

Real-time synchronization would allow timetable updates to be transmitted instantly across all connected devices. Whenever an administrator modifies a timetable entry, the change would appear immediately for students and faculty members without requiring page refreshes.

Benefits of real-time synchronization include:

- Instant schedule updates.
- Reduced communication delays.
- Improved timetable accuracy.
- Better coordination among departments.

For example, if a faculty member is unavailable and another instructor is assigned, the change could be reflected instantly. Similarly, room changes or emergency timetable adjustments could be communicated to all users in real time.

This feature is particularly useful in large educational institutions where schedule changes occur frequently. It would eliminate confusion caused by outdated timetable information and ensure that everyone always has access to the latest schedule.

Real-time synchronization also creates opportunities for future collaborative features, such as faculty schedule management and administrative approval systems.

12.5 Cloud Database Integration

The current version of the timetable viewer stores timetable information using structured JavaScript objects. While this approach is suitable for a front-end project, future scalability would benefit greatly from cloud database integration.

A cloud database would provide centralized storage for:

- Timetable records

- Faculty information
- Classroom data
- Attendance records
- User preferences
- Notification settings

Popular cloud solutions such as Google Firebase, Amazon Web Services, or MongoDB could be used for this purpose.

Benefits of cloud database integration include:

- Real-time data updates.
- Secure data storage.
- Multi-user support.
- Backup and recovery.
- Scalability for large institutions.

With a cloud database, timetable administrators could update schedules from a central management panel. Students and faculty would then receive updated information instantly through synchronization services.

Cloud integration would also support advanced analytics, attendance monitoring, and historical timetable tracking. As the number of users grows, the system could scale efficiently without significant performance issues.

This enhancement would represent a major step toward transforming the timetable viewer into an enterprise-level academic management solution.

12.6 Mobile Application Development

Although the current timetable viewer is responsive and mobile-friendly, a dedicated mobile application would provide a more optimized and feature-rich experience. Mobile applications are increasingly preferred because students spend a significant amount of time using smartphones for academic activities.

A future mobile application could be developed using technologies such as:

- React Native
- Flutter
- Kotlin
- Swift

The application could provide:

- Offline timetable access.

- Push notifications.
- Faster performance.
- Mobile-specific user interface.
- Device-level integrations.

Students would be able to access schedules instantly without opening a browser. The app could also integrate directly with mobile features such as notifications, calendars, location services, and biometric authentication.

Faculty members could use the mobile application to manage schedules, monitor attendance, and receive updates while moving between classrooms.

The development of a dedicated mobile app would improve accessibility and increase user engagement, making the timetable system more convenient and practical for everyday use.

12.7 Machine Learning-Based Scheduling

Machine Learning (ML) has the potential to revolutionize timetable generation and optimization. While the current system focuses on timetable viewing and management, future versions could use machine learning algorithms to assist in timetable creation.

Machine learning models could analyze historical scheduling patterns and automatically generate optimized timetables based on:

- Faculty availability.
- Classroom capacity.
- Subject requirements.
- Student enrollment data.
- Resource constraints.

Benefits of ML-based scheduling include:

- Reduced manual effort.
- Conflict-free timetable generation.
- Better resource utilization.
- Improved academic planning.

For example, the system could automatically detect scheduling conflicts and suggest alternative arrangements. It could also predict classroom demand and recommend room allocations based on attendance trends.

Another possibility is predictive analytics. Machine learning models could analyze attendance patterns and identify students who may be at risk of low attendance. The system could then provide recommendations or reminders to help improve academic performance.

As educational institutions continue to adopt intelligent technologies, machine learning-based scheduling could become one of the most impactful enhancements for the timetable viewer.

CHAPTER 13: CONCLUSION

The **College Timetable Viewer using React, HTML, and CSS** was developed as a modern academic timetable management solution to make schedule access easier, faster, and more organized for students and faculty members. The project was designed to address the common problems found in traditional timetable systems, such as poor accessibility, static display formats, difficulty in updating schedules, and lack of search or filter support. The final implementation includes weekly and daily timetable views, subject-wise filtering, faculty and classroom directories, attendance tracking, theme customization, dashboard widgets, responsive design, PWA support, and print/export functionality. The report confirms that the system successfully demonstrates React components, state management, filtering, and responsive web design, which shows that the project achieved its practical goals.

13.1 Summary of Work

The project began with the identification of a real academic problem: students and faculty often face difficulty in accessing updated timetable information in a clear and centralized way. To solve this, a web-based timetable viewer was planned and developed using React.js, HTML5, CSS3, JavaScript ES6, and Node.js. The aim was to build a user-friendly interface that could display schedules in a simple and structured format while also supporting advanced features such as filtering, navigation, attendance tracking, and theme switching. The objective of the project, as stated in the report, was to develop a user-friendly web application that allows students and faculty members to view college timetables efficiently through daily and weekly schedules, subject details, faculty information, classroom allocation, and filtering options.

.

During development, timetable data was organized into structured objects so that it could be rendered dynamically in the interface. React components were used to display the dashboard, weekly timetable, daily timetable, subject filters, faculty details, classroom data, and attendance tracker. This component-based design made the application modular and easier to manage. The system also included responsive CSS styling with glassmorphism effects, custom themes, and print-friendly output. According to the report, the application was built with features such as weekly and daily views, subject filtering, faculty and classroom filtering, day navigation, search functionality, dark/light theme support, dashboard overview, attendance tracker, PWA support, and PDF/print export.

The source code and logic behind the project were developed in a way that allowed the timetable data to be displayed and updated dynamically. The report mentions that timetable information such as subjects, faculty names, classrooms, timings, and category colors is stored in centralized structured objects inside `timetableData.js`, while `App.jsx` and `styles.css` manage the interface and visual presentation. This approach helped keep the project organized and scalable.

Overall, the work completed in this project covered the full lifecycle of a front-end academic application: problem identification, requirement analysis, system design, implementation, testing, and final output generation. The result is a timetable viewer that is practical for everyday use and also suitable as a strong academic project demonstration.

.

13.2 Achievements of the Project

The project achieved several important goals successfully. The first major achievement was the creation of a centralized timetable viewing system that provides both weekly and daily schedule formats. This makes the application useful for users who need either a complete week overview or a focused daily plan. The weekly view allows students and faculty to visualize the entire schedule, while the daily view makes it easier to focus on one day at a time. The report's output screens confirm that both weekly and daily timetable views were implemented successfully.

Another major achievement was the implementation of search and filter functionality. The system allows users to search for subjects, filter by faculty, and look up classroom information quickly. This feature significantly improves usability because users no longer need to manually scan the full timetable. The report shows output screens for subject filter functionality and faculty/classroom filters, which confirms that these tools were successfully integrated into the system.

A further achievement was the development of a modern and visually appealing user interface. The application includes dashboard widgets, dark theme support, glassmorphism styling, and responsive layouts that make the system look professional and easy to use. The report explicitly mentions premium frosted glassmorphism elements, micro-animations, and custom themes such as Violet, Emerald, Cyberpunk, and Amber. These elements show that the project is not only functional but also aesthetically polished.

The attendance tracker is also an important achievement because it adds academic value beyond timetable display. It helps students monitor their class attendance, calculate progress, and predict how many classes are required to reach a target attendance level. This turns the application into a more complete academic support tool. The report describes this feature as an attendance tracker with subject increments, status progress indicators, and a goal calculator for a 75% target.

.

The project also achieved responsive and browser-friendly behavior. It was designed using modern web technologies that work across common browsers and different screen sizes. The report lists React.js, HTML5, CSS3, Node.js, and browsers such as Chrome, Edge, and Firefox as the software environment. This shows that the application is built for practical use on multiple devices.

.

Another noteworthy achievement is the inclusion of PWA support and print/export functionality. Offline caching and manifest support improve accessibility, while the PDF/print exporter allows users to generate timetable sheets in a readable landscape format. These additions make the system more useful in both online and offline situations.

.

In short, the project successfully achieved its core goals: it provides an efficient way to view, search, and filter class schedules, while also offering a modern interface and useful academic tools. The final result section of the report clearly states that the College Timetable Viewer was successfully developed using React, HTML, and CSS and provides a clean, responsive, and interactive interface for students and faculty to access timetable information efficiently.

.

13.3 Learning Outcomes

This project provided valuable learning experience in both technical and design aspects of web development. One of the biggest learning outcomes was understanding how to build a real-world application using React. The project showed how components can be organized, how state can be managed, and how the user interface can update dynamically without reloading the page. Working with React helped strengthen knowledge of component reusability, event handling, conditional rendering, and data-driven UI development. The report's logic section confirms that the application uses React components to render timetable information, apply filters, manage navigation, and switch themes.

Another important learning outcome was gaining practical experience in structuring data for application use. Timetable information had to be stored in a format that could support weekly views, daily views, search, filtering, and attendance calculations. This required careful planning of data fields such as subject, faculty, room, timing, type, and color. Learning how to keep data organized and centralized is an important skill that will be useful in future software projects.

.

The project also improved understanding of CSS3 styling and responsive design. It became clear how layout, spacing, shadows, transitions, and color themes can change the entire feel of an application. Using CSS for dark/light modes, glassmorphism effects, and mobile-friendly layouts helped demonstrate the importance of design in user experience. The report's mention of responsive design, premium UI, and print-media rules shows that styling was not treated as a secondary task, but as a major part of the project.

Another learning outcome was improving logic-building skills. The project included filtering algorithms, timetable mapping logic, attendance calculations, theme switching logic, and export behavior. These functions helped strengthen problem-solving ability and showed how a front-end system can still have meaningful algorithmic structure. The logic described in the report, including loading timetable data, applying filters, and toggling themes through HTML attributes and CSS variables, reflects this learning.

.

In addition, the project improved understanding of user interface planning. A timetable viewer must be simple enough to use quickly but detailed enough to show subject, faculty, classroom, and attendance information. Designing such an interface required balancing clarity, spacing, hierarchy, and accessibility. This learning experience will be helpful in future projects that require both technical correctness and visual quality.

.

The project also provided exposure to practical deployment concerns such as browser compatibility, print-friendly views, and offline support. These are important real-world issues that go beyond basic coding and help prepare for larger software development tasks. Overall, this project served as a complete learning platform for front-end development, interface design, data handling, and academic application planning.

The **College Timetable Viewer using React, HTML, and CSS** is a successful and useful academic application that meets its intended goals. It improves the way timetable information is displayed, accessed, and managed by replacing static, outdated formats with a modern digital interface. The system provides a centralized and responsive environment where students and faculty can view schedules, search for subjects, filter by faculty or classroom, monitor attendance, and switch between different visual themes. The report confirms that the system successfully demonstrates the use of React components, state management, filtering, responsive web design, and print/export support, which proves that the project is both functional and well-implemented.

The project is important because it addresses a real academic need. Timetable management is a routine but essential part of educational life, and even a small improvement in accessibility can make a major difference in convenience and efficiency. By providing weekly and daily timetable views, search tools, dashboard widgets, and attendance support, the system becomes more than just a timetable viewer; it becomes a practical academic assistance tool. The output screenshots in the report show that the various interfaces were successfully developed and presented in a clean, organized, and visually appealing form.

From a development point of view, the project shows the strength of modern web technologies in solving real problems. React made it possible to build a dynamic and interactive interface, CSS3 enabled a polished and responsive look, and structured JavaScript logic allowed the timetable to function smoothly. Features such as offline caching, theme customization, and print-ready output add further value and show that the system has a strong foundation for future growth.

.

In conclusion, this project was a valuable and successful effort that combined software engineering, interface design, and academic utility in one system. It demonstrates how a well-planned web application can improve daily academic workflow and provide a better experience for students and faculty. The current version already fulfills its main purpose effectively, and the future enhancements mentioned in the report offer a clear path for further development. The project stands as a strong example of a modern, practical, and user-friendly timetable management application built with React, HTML, and CSS.